Detecting IMSI-catchers

The battle against GSM mobile communication eavesdropping

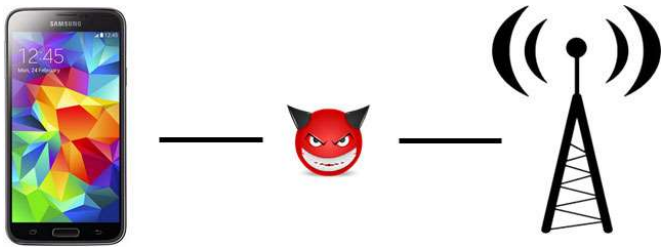
Home
FreeBSD
Education
Technology
Security & Privacy
Business innovation
Finance
Our Universe
Tools
Culture & society
Travel
Health & Life science
Pilot license
Games
Philosophy
Media & arts
Leadership
Lily's recipes
Legacy sites

Home > Security privacy > Detecting gsm eavesdropping imsi catchers

Detecting GSM eavesdropping IMSI-catchers

ANDY CHIU, BEN HUP, FEDERICO FALCONIERI, BAS VAN IJZENDOORN, MARNIX DE GRAAF, PIOTR TEKIELI, CHRISTIAN VEENMAN - 31 MAY 2016 - 5022 - 2 0 - 0

Abstract



Representation/concept of IMSI-catcher

Abstract. IMSI-catchers are problematic due to privacy concerns. The purpose of this paper is to create an IMSI-catcher detection system and scan the city center of The Hague, The Netherlands. The method used comprises of the usage of literature and the use of readily available tools to analyze GSM network traffic. Due to the inherent trusting relationship between cell tower and GSM enabled mobile phones, the phones do what the cell towers tell them to do. This trust relationship worked in the days that the government had a monopoly and operated telecom companies, computer power was low and not yet advanced, but nowadays anyone with a low budget can easily capture broadcasted traffic. In conclusion, we want to know if an IMSI-catcher detecting system for under $20 can be made and whether IMSI-catchers are present in The Hague.

Keywords : IMSI-catcher, eavesdropping, GSM, G2, mobile network, hacking

Table of contents

Table of figures

Table of tables

Summary

GSM networks send voice and data through the air. An unguided medium such as the air allows eavesdropping of communication without sender and receiver knowing that they are eavesdropped. A device capable of eavesdropping GSM communications is an IMSI-catcher. Such devices have become cheaper but can still cost up to $2000. A $20 low budget DVB dongle could be used to pick up GSM

communications. The low budget solution allows for scanning for IMSI-catchers in The Hague around governmental buildings and embassies to detect whether espionage attempts are being made. This report describes a plan for creating a low budget IMSI-catcher and the detection process itself.

In The Hague, judicial capital of the Netherlands, IMSI-catchers can be a threat to national security or at least leak sensitive information. Whenever IMSI-catchers are expensive the risk of anyone having such a device and thus the risk of sensitive information leaking remains low. However, when IMSI-catchers become cheap anyone could own one and the very communication system the world relies on for mobile voice and data could become compromised. In 2014 90% of the world population relies on GSM G2, operating in 219 countries. The impact of a low budget IMSI-catcher has far reaching effects.

Based on primary sources, reports and expert knowledge this report explains how a $20 DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle can be used in cooperation with Ubuntu OS, grgsm and some scripts. The result is an IMSI-catcher that is capable of detecting IMSI-catchers based on four giveaways. The giveaways are: encryption usage, reject messages, empty neighbouring list and deviating signal gain. The implementation allowed for scanning a dozen locations in The Hague near governmental buildings and embassies, short POI's (i.e. Points of Interest).

Giveaways were found in 9 cell towers. Finding the giveaways does not guarantee that the cell towers are IMSI-catchers, especially since the most important giveaway (i.e. disabled encryption) was not found in any tower. Disabled encryption is the indicator with the greatest importance that a cell tower is not operating as expected by Dutch GSM G2 standards.

We believe that one cell tower in particular must either be an IMSI-catcher or a misconfigured tower.

Creating a low budget IMSI-catcher is feasible with the help of open source software and an understanding of GSM G2. This report concludes that at least one IMSI-catcher is present in The Hague, with reasonable indication that an additional eight IMSI-catchers are present, making the total nine possible IMSI-catchers found.

1 Introduction

An IMSI-catcher is an eavesdropping device for mobile phone communications. By creating a fake mobile tower GSM enabled phones connect to the fake tower allowing the eavesdropper to execute a man-in-the-middle (MitM) attack. An IMSI is the International Mobile Subscriber Identity of a mobile phone in essence allowing to uniquely identify a mobile phone in a telephone network.

The objective of this report is to create a procedure to detect IMSI catchers. Both the creation of the IMSI catcher solution and the findings of scanning for IMSI-

catchers in The Hague are reported.

The problem statement is that eavesdropping on GSM users without consent is an illegal activity by law. Nevertheless, empirical evidence has shown that eavesdropping takes place on a federal level by the United States FBI (Soghoian, 2014). Furthermore, security company GSMK detected with its CryptoPhone no less than 18 IMSI-catchers in Washington D.C. (Timberg, 2014).

The purpose is to report about the creation of a software-based IMSI-catcher detector and identify cell towers in The Hague, The Netherlands near Points of Interest (POI) like embassies, national governmental buildings and highly populated areas. The scope is limited to The Hague city centre.

The methodology used involves first reading (scientific) papers, journals and periodical as primary sources for this report. Second, building the IMSI-catcher detector is done using Ubuntu 15.10 64-bit and the grgsm suite.

The contents of the report can be divided in six parts. First, the methodology is explained that shows how creating the IMSI-catcher was setup. Second, the hardware prerequisites and GSM G2 frequency bands are explained. Third, the IMSI-catcher setup used in this report is explained. Fourth, the giveaways that are used in the IMSI-catcher are described. Sixth, the route and significance of locations to scan are explained. Seventh, the scanning results are elaborated upon. Finally, a conclusion summarizes the findings the the project.

2 Methodology

In order to obtain the necessary information for discussion and to obtain a thorough understanding of IMSI-catchers the research is based on primary sources and websites that explain technicalities. The initial search indices are directly related to IMSI-catchers and GSM 2G. Examples of initial search indices are 'GSM', 'IMSI' and 'IMSI catcher'. The results from the searches based on the search indices are used following the *snowball principle* (Verschuren & Doorewaard, 2010). The snowball principle involves searching for new literature using relevant references from already found literature and other sources. The literature is deemed relevant when a paper or other sources answers questions about IMSI-catchers and GSM G2 and further extends the team's understanding.

The theoretical approach is simultaneously executed with tinkering with the DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle, software grgsm and Ubuntu. This approach allows for 'learning by doing' and 'learning by using' (Hekkert M., 2007) and extends the practical understanding of how components and software relate to each other.

3 Hardware prerequisites and limitations

This chapter describes hardware prerequisites in relation to the GSM 2G protocol. A first hunch in selecting hardware was a DVB-T TV Tuner USB dongle using the RTL2832U chipset. Such dongles are available for around $20 per unit. In specific the "DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle" was chosen for this project based on two criteria. First, positive experience of Prof. Dr. Christian Doerr of Delft University of technology and users expressing satisfactory results on on-line fora. And second, the chipset allows for scanning the full GSM-900 frequency bands with relative high precision concidering the cost of the unit. Alternatives like the SDRPlay ($149), Airpsy ($199), HackRF ($300) and BladeRF ($650) are disregarded as an option, because the cost of these units far exceed the budget restrain of $20 (RTL-SDR.com, 2016).

Different chipsets support different frequency ranges. The "Rafael Micro R820T" chipset both has the widest frequency range and the cost remains within the $20 budget constraint as seen in Table 1.

Table 1 Different chipset properties; Adapted from (RTL-SDR.com, 2016)

	Tuner	Frequency range	Cost
1.	Elonics E4000	52 – 2200 MHz with a gap from 1100 MHz to 1250 MHz (varies)	$ 40
2.	Rafael Micro R820T	24 – 1766 MHz (with experimental drivers: 13 – 1864 MHz)	$ 22
3.	Fitipower FC0013	22 – 1100 MHz	$ 15
4.	Fitipower FC0012	22 – 948.6 MHz	?
5.	FCI FC2580	146 – 308 MHz and 438 – 924 MHz (gap in between)	?

The required GSM G2 frequencies of interest involve the E-GSM-900 frequencies between 880.0 – 915.0 MHz as uplink (mobile to base) and 925.0 – 960.0 MHz as downlink (base to mobile). GSM band properties can be viewed in Table 2. Different GSM G2 frequency bands are proposed for different usages:

1. P-GSM, Standard or Primary GSM-900 Band

2. E-GSM, Extended GSM-900 Band (includes Standard GSM-900 band)

3. R-GSM, Railways GSM-900 Band (includes Standard and Extended GSM-900 band)

4. T-GSM, Trunking-GSM

Table 2 GSM bands and their properties; Source: (ETSI, 2005)

	GSM band	ƒ (MHz)	Uplink (MHz) (Mobile to Base)	Downlink (MHz) (Base to Mobile)	Channel number
1.	P-GSM-900	900	890.0 – 915.0	935.0 – 960.0	1 – 124
2.	E-GSM-900	900	880.0 – 915.0	925.0 – 960.0	975 – 1023, 0 - 124
3.	R-GSM-900	900	876.0 – 915.0	921.0 – 960.0	955 – 1023, 0 - 124
4.	T-GSM-900	900	870.4 – 876.0	915.4 – 921.0	dynamic

The "Rafael Micro R820T" chipset has a standard frequency range of 24 – 1766 MHz which covers the frequency range that is needed in this project. The frequency range can be extended to 13 – 1864 MHz with experimental drivers. The price is around $20, the chipset does not show a gap in the frequency range. Also important is that the chipset is widely available and used by many users improving availability of information.

Hardware limitations

Experience and practical understanding in the form of 'learning by doing' and 'learning by using' (Hekkert M., 2007) is an effective way of learning that can bring to light issues not present in any documentation.

Three idiocyncratic behaviours of the "DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle" emerged from using the hardware. First, scanning with different dongles resulted in different frequencies being detected. Escpeccialy the research hallmarks such as replicability, precision and confidence are affected by behavioural differences between dongles. We were unable to correct this problem. Second, a dongle could become unresponsive or otherwise stop functioning at random, unpredictable moments in the form of 'freezing up'. The solution was to disconnect of remove the dongle from the USB socket and reconnect the dongle to the computer. The dongle would function normally after reconnecting. Third, the dongle could become overheated. Although the NooElec Nano and NooElec Nano 2 are known for overheating problems due to fitting the circuitry into a much smaller dongle, the improved NooElec Nano 2+ and the original NooElec R820T2

NESDR Mini2 (the dongle usedn in this project) should be unaffected by heating problems according to user feedback at for example Amazon.com. The overheating could also be a cause for an unresponsive dongle. The three mentioned issues resulted in the necessity to recapture data because data was lost when a dongle stopped responding. In the end the team persisted in getting the data needed to finish the project.

4 Setting up an IMSI-catcher detector

This chapter elaborates on how to set-up and use an IMSI-catcher-catcher (i.e. a device that catches IMSI-catchers). To do this, a description on how to install all used tools is given in paragraph 4.1. This is followed by paragraph 4.2 in which these tools are combined to make an IMSI-catcher-catcher. This chapter ends with paragraph 4.3 describing the usage of the produced scripts. Figure 1 provides a high-level organization of the different scripts and which information is exchanged between them in order to find IMSI-catchers.

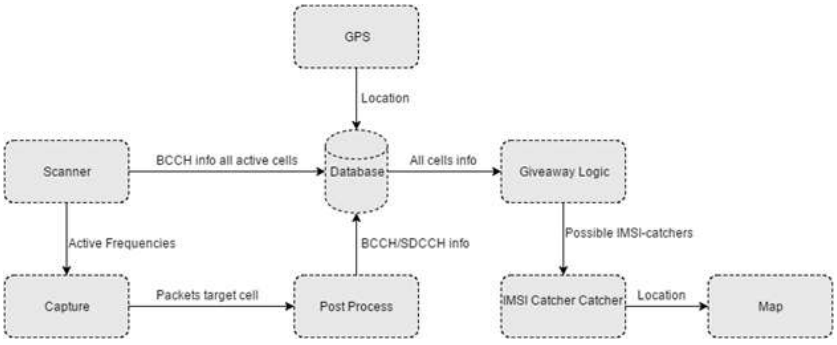


Figure 1 IMSI-catcher detector information flow

Each element in Fig 1 corresponds to a script, paragraph 4.2 will provide detailed information regarding each script. The main idea is to collect different types of information about base stations that are within range using the available tools, all this information will be centralized in one database. From this database we are able to perform queries and see whether any of the giveaways are present. Finally, the possible IMSI-catchers are printed out and displayed on a geographical map.

4.1 Installing readily available tools

Several readily available tools combined can be used to build an IMSI-catcher-catcher. Aside from an antenna as capturing hardware, software is needed to save raw GSM data and to decode this into pre-defined readable packets. Next, software is wanted to read into these packets and process the useful data which can be analysed later. For this IMSI-catcher-catcher the following software is used:

1. VMware (or other virtualisation software); to be able to run the same configuration on every used machine (see "Appendix 1: Installing Ubuntu 15.10 64-bit")

2. GNUradio (see "Appendix 3: Installation of gnuradio"); used in GR-GSM as software defined radio to 'understand' GSM traffic

3. GR-GSM (see "Appendix 4: Installation of grgsm")

a. Gr-gsm livemon; can scan and decode live gsm traffic. Used to test if frequencies actually work correctly.

b. Gr-gsm scanner; used to scan for and give basic data of available frequencies. A small contribution was made to the source code of Gr-gsm scanner to correct faulty code used in this project.

c. Gr-gsm capture; used to capture raw GSM data

d. Gr-gsm decode; used to decode the output of the capture into readable GSM-packets

4. Wireshark (and Tshark); to be able to read into and filter the captured gsm-packets (see "Appendix 5: Installation of Wireshark")

5. Python-pcapng; can parse Tsharks output (which are pcapng files) to be able to access packet information from within the scripts (see "Appendix 13: post_processing.py")

After installing these software packages it is possible to scan for and save data of some active frequency. This is not enough to detect IMSI-catchers yet. The next section describes how this can be achieved by connection the mentioned tools.

4.2 Connecting the software

The separate software tools from the last section can be used together as an IMSI-catcher-catcher. The general steps to achieve this are as follows:

1. Search for available active GSM-frequencies. These differ per GSM-cell.

2. Capture raw GSM data from (all of) these frequencies and decode this data into readable packets

3. Filter useful package data and save this to a database containing information per GSM-cell

4. Process the database on IMSI-catcher giveaways (see chapter 5) and present results

Step 1
The code for step 1 can be found in "Appendix 7: imsicc_scanfreq.pl". A database is created which contains the information per GSM-cell. The grgsm_scanner

application is used to scan for available frequencies and also retrieve other information such as nearby tower frequencies, signal strength, cell id and location and provider codes. This information is inserted in the database.

Step 2

In "Appendix 12: main.py" the code can be seen for a script which takes care of step two. First, a folder is created per capture and the frequencies gathered in step 1 are checked before capturing data using grgsm-livemon. This is done because during capturing it was noticed that sometimes a frequency is not returning any data. When scanning on one capture device, and capturing data on another it was found that the devices are not tuned to exactly the same frequency. Also, some devices worked better than others resulting in one device being able to receive a frequency, while the other is not. To make sure that we only save and process 'good data' the extra check is performed and only working frequencies are captured.

To increase capturing speed, the script allows for using multiple devices at the same time. This was not used in the end because the hardware seemed not reliable enough. The script now captures every accepted frequency for a given amount of time using grgsm_capture and when finished starts decoding this file using grgsm_decode. The output of the decode is captured by Tshark which saves it. Another capture is started in the background. Opposed to capturing; only one decode can happen simultaneously as otherwise Tshark saves data from multiple towers in one file, which is not wanted for later steps. The decode is performed directly instead of later because the capture with grgsm_capture produces a lot of data (about 1 GB per minute), of which a very big part can be thrown away directly as it is coded. For detecting the IMSI-catcher, only broadcast data from the tower is needed and not the actual gsm-data packets which contain conversations and internet-data.

Step 3

After completing step two the useful broadcast data from towers is saved as pcapng files, containing the GSM packets. These can be post-processed for step 3 with python-pcapng, which is used in the script from "Appendix 13: post_processing.py". The script processes all available pcapng files one by one. Per file, a loop is run for which each packet is analysed. Per packet a check is performed which type of packet it is. If the packet is either a System-information-type-3 or ciphering-mode (lapdm) type, then it is further processed. The rest is discarded, but can also be used if the script is extended for more giveaways.

Per type, information is extracted such as the cell id, lac (location area code), mcc, mnc, signal gain, location update requests/accepts/rejects and used cipher. More information on why and how these values are obtained can be found in chapter 5. This information is then added to the database from step 1.

Step 4

The results of the previous three steps is to create one database table where all broadcast data from towers is stored. "Appendix 10: DBMerger (JDK 7)giveaways.py" contains the script that is able to detect four giveaways. These giveaways are related to the used encryption, neighbour list, rejections and signal gain of a cell. A function exists for each giveaway and it returns a list of cell ID and frequency pairs for all cells that show the corresponding giveaway. The final script in "Appendix 11: config.py" uses the lists to create a clear overview of all cells that contain at least one giveaway. All giveaways are shown for a particular cell if it is present, more giveaways mean a higher likelihood of it being an IMSI-catcher. Apart from printing the results in the terminal, it is also stored in the database so that the data can be inspected in a tabular format.

4.3 Using the scripts

The order of using the scripts are described in the last section. The workings of the scripts are determined by a configuration file, of which an example can be found in "Appendix 11: config.py". Here the frequencies can be set, available antenna devices, which channels to decode and file locations. After setting the configuration (using the terminal output of step 1, see Table 3) the scripts can be run in order from the terminal using 'python <scriptname>' when executed from within the scripts folder.

The scripts automate work which can also be performed manually (which is quite time-consuming), so the only thing to do is keep an eye on the terminal output to see if any errors occurred. The scripts are not combined as results needed to be verified during the project. This is also possible to do, but not useful in this case. After performing the steps and scripts, the possible IMSI-catchers are shown in the terminal and displayed on a geographical map.

Table 3 Terminal output grgsm_scanner

```
grgsm_scanner
example output:
ben@ubuntu:~$ grgsm_scanner
linux; GNU C++ version 5.2.1 20151010; Boost_105800; UHD_003.010.git-0-2d68f228

ARFCN:   5, Freq:  936.0M, CID: 11750, LAC:  3150, MCC: 204, MNC:   8, Pwr: -28
ARFCN:  17, Freq:  938.4M, CID:     0, LAC:     0, MCC:   0, MNC:   0, Pwr: -32
ARFCN:  18, Freq:  938.6M, CID: 12170, LAC:  3150, MCC: 204, MNC:   8, Pwr: -27

ben@ubuntu:~$ grgsm_scanner -b R-GSM -v
```

5 Giveaways

An IMSI catcher can be detected by examining GSM packets that are sent through the air. Before being able to capture the GSM packets a study of the GSM protocol and the scientific literature on IMSI catcher giveaways by Dabrowski *et al* (2014) was necessary. From the study, the four most important giveaways were chosen to use

in the proposed IMSI catcher scanning software. The four most important were chosen in consultation with Dr. Christian Doerr of the Cyber Security Group at Delft University of technology. After deliberation and prioritising with Dr. Doerr using his experience, the four selected giveaways are most likely to happen in the Netherlands. Giveaways depending on the comparison of found cells to public databases of clean cells owned by service providers are impractical in the Netherlands. The service providers in the Netherlands do not provide clear lists of the cells they maintain. There is the OpenCell ID (http://opencellid.org/) database available but this database might also contain IMSI catcher cells as it is user generated. Therefore, a valid comparison can't be made. Other giveaways would probably just not occur in the Netherlands.

The four giveaways in order of importance are:

1. BTS force use of no encryption (A5/0) or weak encryption (A5/2) algorithm.

2. A large number of Location Updating Reject messages.

3. Empty neighbouring list.

4. High signal gain

The next subchapters will explain the giveaways in detail.

5.1 Encryption

During the handshake between a Base Station and a Mobile Station, among the many messages exchanged there is one that the Base Station uses to select the encryption algorithm: the Ciphering Mode Command message. This is an important aspect of why GSM is so insecure: the Base Station decides the encryption and the Mobile Station has no say in this. Four algorithms exist to encrypt communication in the GSM protocol: A5/0, A5/1, A5/2, A5/3. Among these, A5/0 does not apply encryption at all and A5/2 is considered cryptographically broken and can be decrypted in real time by an attacker. A5/1 and A5/3 are considered more secure and are the ones that mobile networks are supposed to use in The Netherlands.

An IMSI Catcher, indistinguishable from a real Base Station from the point of view of a Mobile Station, could easily enforce the use of A5/0 to be able to listen to intercepted calls and text messages in real time, without the hassle of decryption. Alternatively, also finding the use of A5/2 should raise suspicion: it is cryptographically broken and mobile networks are not supposed to use it.

The Ciphering Mode Command message travels on SDCCH (in plaintext) and it can be captured with the grgsm suite. This message contains the algorithm that the Base Stations wants to use as a string of text. If during a grgsm capture a Base Station is found to be using A5/0 or A5/2, this is a strong giveaway that the Base Station could actually be an IMSI catcher (Ooi, 2015).

5.2 TMSI rejections

During the GSM handshake, the Mobile Station is assigned a TIMSI by the network through the Base Station. This happens because the IMSI is sensible information, being a unique identifier of a SIM, and therefore it should be hidden as much as possible. The TIMSI remains the same while a user is connected to a specific network because the couple (IMSI, TIMSI) is known by the network and therefore every Base Station of the network will recognise the user as a subscriber. When a user moves to an area where Base Stations of the preferred network can't be found, the phone will connect to a Base Station of another network (what is commonly known as 'roaming') with a Location Updating Request message (on the SDCCH channel). The new network will not recognise the TIMSI and it will send a Location Updating Reject message to the Mobile Station through the Base Station. This will trigger a new handshake between the Mobile Station and the network in which the Mobile Station will be assigned a new TIMSI. Now, if an IMSI catcher is spoofing a certain network but has no access to the database where the couple (IMSI, TIMSI) is stored, it is to be expected to detect a high number of these Location Updating Request and Location Updating Reject messages in its proximity.

Furthermore, if an abnormal rate of handshake failures is detected (the number of Location Updating Rejects messages is significantly higher than the number of Ciphering Mode Command messages), this is an important giveaway that the IMSI Catcher is targeting a specific IMSI or a small subset of IMSIs.

In our program we decided to flag this giveaway if the number of Location Updating Reject messages is above 25% of the BSS replies (as explained before, the BSS can either reply with a Location Update Reject or a Ciphering Mode Command).

```
# = number of messages

if (#ciphering mode command + #location update reject) > 30 {
    if #location update reject > [(#ciphering mode command + #location update reject) * 0.25] {
        -->flag giveaway
    } else{
        -->don't flag giveaway
    }
} else{
    -->don't flag giveaway
}
```

Figure 2 Giveaway logic (Source: (Rahnema, 1993))

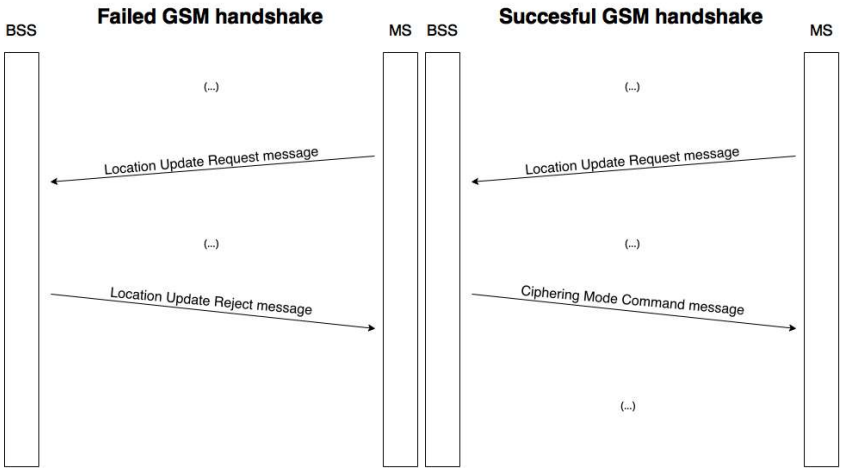


Figure 3 GSM filed and successful Handshake (Source: (Ooi, 2015))

5.3 Neighbour list

Once a Mobile Station (MS) is connected to a cell, it needs to periodically monitor a list of alternative BCCH carriers so that cell re-selection can take place when necessary. This can be when the current cell has a low C1 value, a cell selection criterion, for a long period. Other cases are when the cell is simply unavailable or that the maximum number of unsuccessful retransmission attempts have been exceeded. If a MS finds itself in any of these positions, it needs to be ready to switch to another cell in order to regain access to the GSM network.

All cells maintain a so-called BCCH allocation list (BA List) containing the frequencies supported by neighbouring cells. This list is broadcasted on BCCH together with other control messages so that a connected MS can use this information during the cell selection and cell re-selection process. A MS will continuously make measurements on its neighbouring cells as indicated by the BA list to initiate cell reselection if necessary. An average signal level will be maintained for each carrier in the BA list. It is in the interest of an attacker that has caught a target with an IMSI catcher not to let cell re-selection take place. Since the BA List is transmitted by the cell itself with no way of verifying whether this information is correct, an IMSI catcher could do one of two things:

1. Transmit an empty neighbour list

2. Transmit a list of neighbours that are unavailable

Either way, the MS will not find an active cell with a better signal than the IMSI catcher it is currently connected to. This technique is can be seen as a form of cell imprisonment. The attacker is likely to resort to ways that lock the target to the IMSI catcher if the purpose is to perform a man-in-the-middle attack for as long as possible. Therefore, a manipulated BA List according to the two cases is a strong giveaway that an IMSI catcher is present in the area. Fortunately, simply scanning the active cells on BCCH can already tell if there is a cell that repeatedly transmits an empty neighbour list. Checking whether the BA List solely consists of unavailable neighbours can be done by comparing the list with the active cells found in a certain area. It is to be expected that not all neighbouring cells are visible to a MS, but it would be a giveaway if all of them are unavailable Figure 4 illustrates an ideal situation where each of the eight hexagons represents the broadcast area of different BSs. Coverage is strictly separated and thus each BS will not receive any information regarding adjacent cells. Though in practice, the scenario depicted in Figure 5. Is more likely, especially in a densely populated urban city such as The Hague. BS coverage will overlap and at any location within the city, multiple BSs should be available.

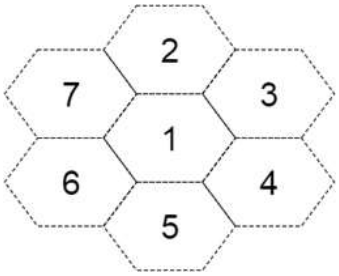


Figure 4 Ideal BS coverage

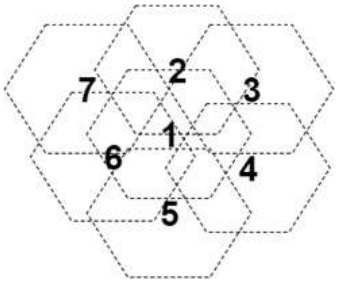


Figure 5 BS coverage in practice

All cells periodically transmit System Information (SI) messages of type 2/2bis/2ter/2quater on BCCH. These contain neighbour cell descriptions and other control messages required for reselection. Furthermore, a message of type SI 5/5bis/5ter provide a similar list, but equal or smaller in size. There are two lists because the MS can scan in either idle or active mode. Scanning when idle can be done on a longer list and tune to the strongest signal. During active mode, the MS is will focus on a smaller subset of the list so that it can get more accurate estimates of the signal strength faster. For this reason, we will mainly focus on the SI messages corresponding with an idle mode, namely SI 2/2bis/2ter/2.

5.4 Signal gain

A mobile phone user can move around and get out of range of its current radio cell. When in idle mode the mobile station (MS) can reselect a cell. The reselection process goes in two phases. In the first phase is a selection made from neighbouring cells who have a higher signal strength than the current cell. The difference in level is expressed in a C1 criterion. The neighbouring cells are ordered according to their C1 value. In phase 2, the perceived signal strength of a cell can be increased by an offset value the base station can broadcast to mobile stations.

An IMSI catcher can exploit this property by setting a high offset value such that mobile stations will pick the IMSI catcher instead of other base stations.

The signal gain is calculated by the MS in the following way. The C2 value represents the new signal strength of a cell. A PENALTY_TIMER is started for each cell in the list of the 6 strongest neighbour cells as soon as it is placed on the list. During the penalty time C2 is calculated as follows:

C2 = C1 + CELL_RESELECT_OFFSET - TEMPORARY_OFFSET

After the penalty time:

C2 = C1 + CELL_RESELECT_OFFSET

Before penalty time:

C2 = C1 - CELL_RESELECT_OFFSET.

Due to fading effects, the strengths might change quickly. Therefore, a CELL_RESELECT_HYSTERIS is added to C1 and C2 of the current cell to reduce the number of reselects. An attacker can make the hysteresis very high such that phones connected to the IMSI catcher stay connected. It can also make the hysteresis very low to get more MS connected to it. An attacker can also change the hysteresis over time to get more control over the reselection of a MS. For instance, it might lure MS to it with a low hysteresis and then keeps them by making the hysteresis higher.

The CELL_RESELECT_OFFSET, TEMPORARY_OFFSET and CELL_RESELECT_HYSTERIS are set in the BCCH System information type 3 and System information type 4. The default values of both offsets are 0 and the default value of the hysteresis is 2. If the values differ from the defaults there might be an IMSI catcher manipulating the reselection process (Stuhlfauth & Schwarz, 2015).

6 Scanning in The Hague

The Hague has a large number of targets of high interest for GSM espionage in a rather small area. Targets fall in two main subsets: government buildings (ministries, the parliament) and foreign embassies.

Communications in and around governmental buildings, either by employees or cooperating partners, could be of interest to other (malicious) parties. Radio communications are unguided and readable outside governmental buildings. Because of the sensitivity and ease of catching, governmental buildings are an ideal target for cell phone interception. Embassies were chosen because of the possibility of being a trans-national victim concerning gsm espionage or conducting espionage operations themselves. Among the documents released by

Edward Snowden in 2013, some reported that the NSA was spying on Chancellor Angela Merkel's cellphone from the US embassy (Der Spiegel, 2013).

The locations where we captured data are the following:

1. Buitenhof. Intended target: Embassy of Israel.

2. Lange Voorhout. Intended targets: British and Swedish embassies.

3. Korte Voorhout. Intended targets: American Embassy, Ministry of Finance

4. Zeestraat. Intended targets: Russian, Ukranian and Turkish embassies.

5. Plein 1813. Intended targets: Canadian Embassy, official residences of many ambassadors.

6. Turfmarkt. Intended targets: Den Haag central station, Ministry of the Interior and Kingdom Relations, Ministry of Security and Justice, Ministry of Social Affairs and Employment, Ministry of Education, Culture and Science.

A capture attempt was made in the Parliament square, Binnenhof, but we were stopped and questioned by the security.

A capture at each location took approximately 45 minutes to complete. Because the captures drained the batteries of the laptops, several days were necessary to complete all proposed captures. The capture locations are shown on the map in Figure 6. Green points are embassies, red points are government buildings, and yellow points are locations where we captured data.

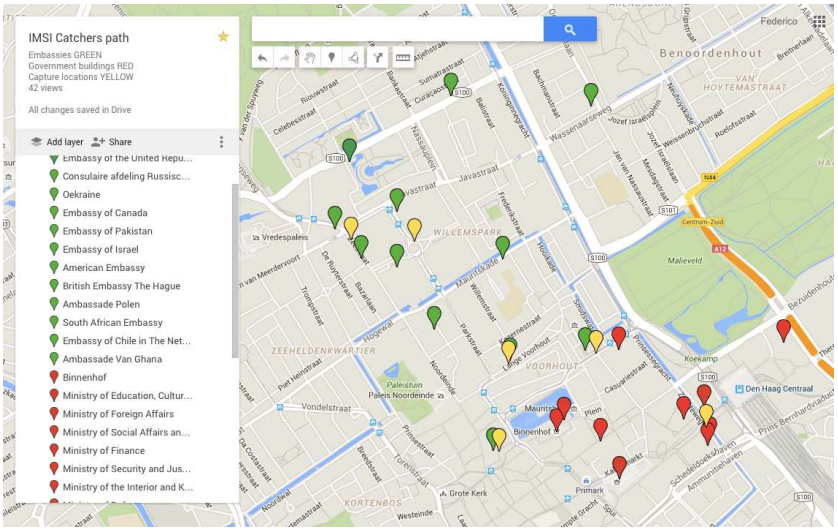


Figure 6 Scanning for IMSI catchers locations; Green = embassies, Red = governmental buildings, Yellow = capture locations

7 GPS Tagging

During the preparation phase, it has been decided that the information captured during each scanning phase has to be complemented with a respective entry indicating where the operation was performed. The purpose of such agreement was dictated by the need for developing a reliable map of IMSI catchers and estimating their positions with a triangulation process. Since a large part of modern community has access to mobile devices equipped with GPS modules (mainly mobile phones) and advanced operating systems, such as Android or IOS, we have decided to use this fact in our research and prepare an application Figure 7 that would allow user to download GPS data using such portable solution. In the cooperation with a publicly available free version of *ShareGPS* program, the relevant NMEA strings were recovered and uploaded to research database. The detailed information regarding used software components, the way they were configured, utilised and/or developed can be found in appendix "Appendix 9: GetGPS (JDK 7)".

The triangulation process is described in chapter 8.

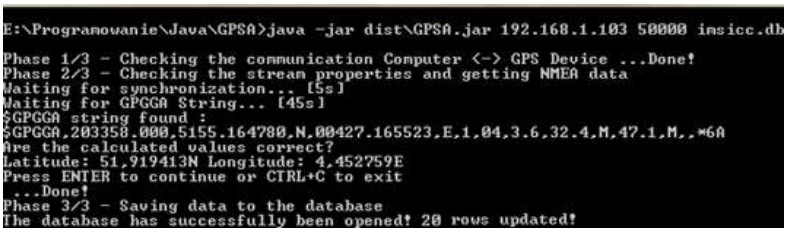


Figure 7 GetGPS v1.7, GPS location tracking

8 Triangulation

To obtain a Tower's location we need to use triangulation. Triangulation is based on the intersection of three circles. Every measurement that we take consists of the tuple <IMSI, Signal Strength, Location, Frequency>. Based on these value a circle can be drawn at the location of the measurement. The boundary of the circle represents the possible locations on which the tower could be located. If we travel to another location and do another measurement of the same tower another circle can be drawn. The intersection of these two circles then intersect at two points and therefore reduces the possible locations of a tower to two. A third measurement is necessary to obtain the point on which all three circles intersect with each other.

Reflection in cities

Triangulation would work perfectly in a world without obstructions or distortions of the signal strength. Unfortunately, in city-like areas the signal is often disrupted by the large amounts of buildings. Because of this, all three circles often do not intersect at a single point and thus hinders us in determining the perfect location. Because of this, the measurement locations are often added up (with their signal strength as a weight) and then averaged to obtain an estimate of the location of the tower.

9 Results

The scanning effort detected 9 possible IMSI catchers in The Hague at the following locations:

Figure 8: Sample terminal output of the results

Figure 8 shows a sample of the output when running the IMSIcatcher.py script. It prints out all relevant information regarding each of the possible IMSI catchers. Only cells for which at least one giveaway is detected will be considered as an IMSI catcher. More giveaways increase the likelihood of a cell being an IMSI catcher. Apart from printing out the results, the data is also stored in the database as shown in Figure 10.

To aid in locating the IMSI catchers we created a web application with Google Maps that is able to calculate the location of those IMSI catchers by providing 3 or more measurements at different locations. Unfortunately, out of the 9 possible IMSI catchers, only 3 of them were captured at 3 or more different locations and therefore we were only able to display 3 of the 9 towers. The web application is programmed as such that it only displays the measurements (Green markers) of towers (Red markers) that have at least 3 measurements associated with them. This way calculation the tower's location does not require manual calculation anymore but can instead be easily calculated and displayed by the web application. The radius is currently being calculated by using the largest distance to each measurement. This might be improved in the future to retrieve more accurate results. Figure 9 shows the location of the 3 possible IMSI catchers with at least 3 measurements associated with them.

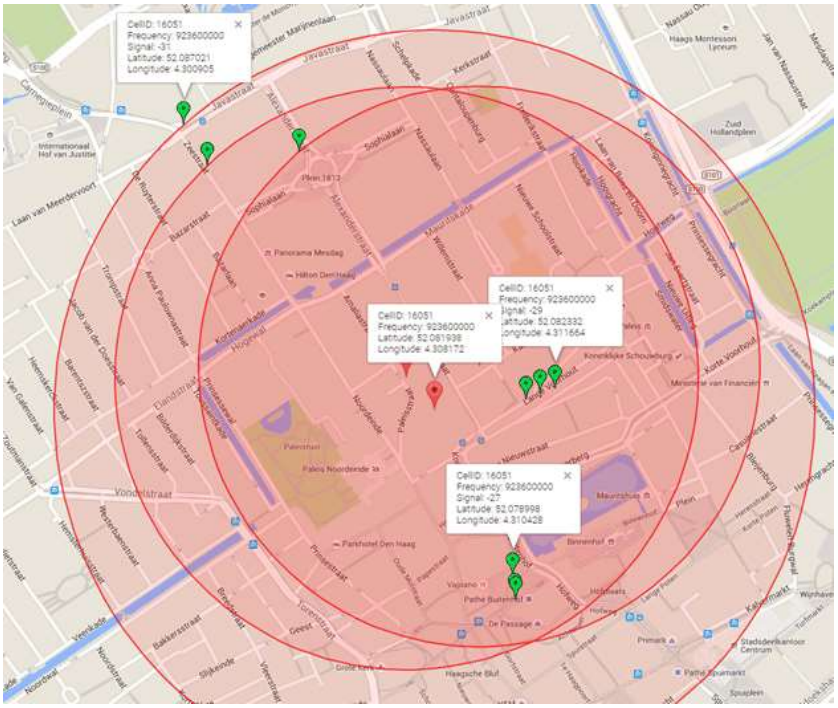


Figure 9 Possible IMSI Catchers locations displayed on a map.

	id	cellid	frequency	encryption	neighbourlist	signal_gain	rejections
1	1	12562	932400000		5		
2	2	61758	928800000		1		
3	3	533	930400000		1		
4	4	12492	933600000		1		
5	5	573	934400000		1		
6	6	62617	928200000		1		
7	7	563	934800000		1		
8	8	16051	923600000			0 1 2	
9	9	16063	924200000			0 1 2	

Figure 10 Database table of the results

The four detectable giveaways are encryption, neighbour list, signal gain and rejections as described in chapter 5. If a value is present in the column for a corresponding giveaway, then it means that the giveaway is present. The value itself provides additional information about the giveaway. For instance, the encryption used will be shown when it is considered weak or not available at all. Sadly, this important giveaway was not detected with any of the scanned cells. The value of the neighbour list column shows the number of different locations where

the cell has been scanned. Due to the unreliable scanning equipment, it is not uncommon to miss a few towers during a scan. A high number makes the cell much more suspicious. The entry for the signal gain contains three numbers corresponding to the reselect offset, temporary offset and reselect hysteresis respectively as described in Section 5.4. Finally, the counted messages regarding the rejections giveaway are also provided when the giveaway is present. These are the following types of messages: location update requests, location update rejections, ciphering mode commands as described in section 5.2.

In the first of the 9 cells listed in figure 10 no neighbours have been found after 5 scanning attempts at different locations. This is abnormal in an area like central Den Haag and we can conclude that this tower is either very poorly configured or an IMSI-catcher.

10 Conclusion

The purpose of this report is to create an IMSI-catcher-catcher (i.e. a device that catches IMSI-catchers) for a budget around $20, to scan Points of Interest in The Hague and identify IMSI-catchers. The IMSI-catcher is based on DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle hardware, available for around $10 to $20. Instead of redoing work that has already been done by other interested parties, we reused software and connected these software parts together via scripts to fulfil the needs for this project. The result is a product that is capable of detecting IMSI-catchers via four giveaways. First, the use or lack of encryption. Second, the number of Updating Reject messages. Third, unusually high signal gain usage. And finally, empty neighbouring lists.

At some locations, multiple scans are executed to further substantiate whether a cell tower is an IMSI-catcher. Using the data collected in the field this study claims that at least 8 towers provide some indication of being an IMSI-catcher, because a giveway has been detected. However, the detected giveaways are considered weak. The probability remains small when the weak giveaways are combined with unreliable equipment and lack of data. Giveaways about one cell tower do present a strong indiciation of that cell tower being an IMSI-catcher. However, it must be noted that less ulterior circumstances are possible like the tower simply being misconfigured. This doubt could be resolved by further tests as well as interviewing the network provider that owns the cell tower.

The project was also motivated by creating an IMSI-catcher on a low budget (i.e. around $20). This report concludes that such a low budget IMSI-catcher can be created. Such an IMSI-catcher uses open source operating system Ubuntu and open source software like grgsm. For positioning, any GPS enabled device, including a phone, can be used as the GPS coordinates can be readily retrieved as plain text. The different hardware and software components are connected by Perl and Python scripts.

As it is often the case, cheap hardware comes with drawbacks. While working with the DVB-T+FM+DAB 820T2 & SDR USB 2.0 dongle we run into three main problems. First: overheating. During captures the dongles would consistently become very hot in a matter of minutes. Second: reliability. More often than not the dongle would randomly stop working. Maybe due to the overheating aforementioned, dongles would suddenly stop working. In some instances, this meant having to re do complete captures. Third: consistency of results with different dongles. Different dongles on the same machine in the same place and time would consistently find different frequencies. This had severe implications on our attempts to parallelize collection with more dongles at once on the same machine.

Future work and research can be done to include more giveaways and to further automate the process of IMSI-catcher detection. However, this project and report are proof that a low budget IMSI-catcher can be created and is able to detect IMSI-catchers.

11 References

Dabrowski, A., Pianta, N., Klepp, T., Mulazzani, M., & Weippl, E. (2014). IMSI-catch me if you can: IMSI-catcher-catchers. *ACSAC '14 Proceedings of the 30th Annual Computer Security Applications Conference* (pp. 246-255). New York: ACM.

Der Spiegel. (2013, October 27). *Embassy Espionage: The NSA's Secret Spy Hub in Berlin*. Retrieved from Der Spiegel: http://www.spiegel.de/international/germany/cover-story-how-nsa-spied-on-merkel-cell-phone-from-berlin-embassy-a-930205.html

ETSI. (2005). *Digital cellular telecommunications system (Phase 2+); Radio Transmission and Reception (3GPP TS 05.05 version 8.20.0 Release 1999).* Sophia Antipolis Cedex, France: ETSI. Retrieved May 12, 2016, from http://www.etsi.org/deliver/etsi_ts/100900_100999/100910/08.20.00_60/ts_100910v082000p.pdf

Hekkert M., S. R. (2007). Functions of innovation systems: A new approach for analysing technological change. *Technological Forecasting & Social Change 74(4)*, pp. pp .413-432.

Mehaffey, J. (2016, 04 28). *NMEA Data*. Retrieved from Joe and Jack's GPS Information Website: http://www.gpsinformation.org/dale/nmea.htm

Ooi, J. (2015). *IMSI Catchers and mobile Security.* Philadelphia: University of Pennsylvania, School of Engineering and pplied Science.

Rahnema, M. (1993, April). Overview Of The GSM System and Protocol Architecture. *IEEE Communications Magazine, Vol.31, Issue 4*, pp. 92-100. doi:http://dx.doi.org/10.1109/35.210402

Rettig, M. (2013, August 3). *rtl_test error message*. Retrieved March 28, 2016, from groups.google.com: https://groups.google.com/forum/#!topic/ultra-cheap-sdr/6_sSON94Azo

RTL-SDR.com. (2016, May 12). *About RTL-SDR*. Retrieved from RTL-SDR.com: http://www.rtl-sdr.com/about-rtl-sdr/

Soghoian, C. (2014). Your secret StingRay's No secret anymore: The vanishing government monopoly over cell phone surveillance and its impact on national security and consumer privacy. *Harvard Journal of Law & Technology, Volume 28, Number 1 Fall 2014* , 34.

Stuhlfauth, R., & Schwarz, R. &. (2015). *GSM and GPRS System Information.* Munich: Training Center Munich. Retrieved from http://read.pudn.com/downloads161/ebook/733566/GPRS/GPRS_chap12.pdf

Timberg, C. (2014, September 17). *Tech firm tries to pull back curtain on surveillance efforts in Washington* . Retrieved march 28, 2016, from The Washington Post, National Security: https://www.washingtonpost.com/world/national-security/researchers-try-to-pull-back-curtain-on-surveillance-efforts-in-washington/2014/09/17/f8c1f590-3e81-11e4-b03f-de718edeb92f_story.html

Verschuren, P., & Doorewaard, H. (2010). *Designing a Research Project.* The Hague: Eleven International Publishing.

Appendix 1: Installing Ubuntu 15.10 64-bit

Installation of gnuradio and grgsm should be possible under any POSIX or Unix-like operating system. However, grgsm is most easily installable under Ubuntu.

Installation of grgsm under FreeBSD 10.1 64-bit requires at least disabling some lines of code in the header files and even still the problem of not compiling persists.

The solution chosen is using Ubuntu 15.10 64-bit. The rest of the appendices build on the assumption of using this operating system. Because of a bug in Ubuntu 15.10 64-bit in the operating system update sequence it is strongly discouraged to run the update process after a fresh installation when asked by the operating system.

For practical reasons VMWare 11 can be used, but Ubuntu 15.10 64-bit can also be installed on the physical hardware itself.

Ubuntu 15.10 64-bit can be downloaded here:

http://www.ubuntu.com/download/desktop/thank-you?country=US&version=15.10&architecture=amd64

Intel i5-5675C processors have a bug that makes VMWare 11 crash when installing or using Ubuntu 15.10 64-bit. Windows crashes giving a Blue Screen of Death (BSOD) with message machine_check_exception.

To circumvent this problem three steps are necessary:

1. A BIOS update with specific microcode update for the motherboard

2. Disable SpeedStep in the BIOS

3. Disable Turbo Boost in the BIOS

Appendix 2: Installing the dongle

Installing dongle commands:

```
$ sudo apt-get install git
$ sudo apt-get install cmake
$ sudo apt-get install libusb-1.0-0.dev
$ sudo apt-get install build-essential

$ git clone git://osmocom.org/rtl-sdr.git
$ cd rtl-sdr
$ mkdir build
$ cd build
$ cmake ../ -DINSTALL_UDEV_RULES=ON -DDETACH_KERNEL_DRIVER=ON
$ make
$ sudo make install
$ sudo ldconfig
$ sudo cp ../rtl-sdr.rules /etc/udev/rules.d/
```

Now reboot Ubuntu.

After reboot do the following:

```
$ sudo rtl_test –t
```

After entering the sudo password you may get the following message when running Ubuntu in VMWare:

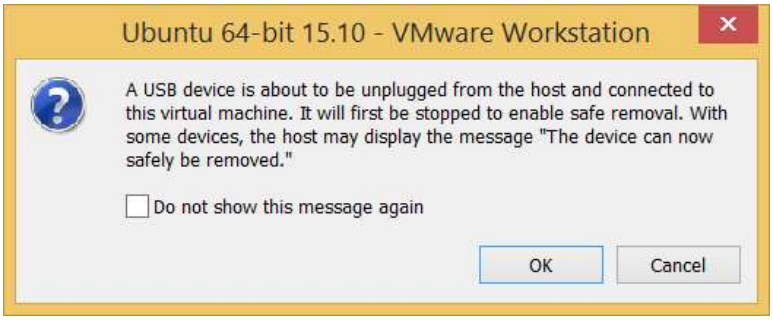
No supported devices found.

The solution to this problem is:

1. Right click on the USB dongle icon in the bottom right corner of Ubuntu

2. Left click “Connect (Disconnect from Host)”

3. Agree by left clicking ok if a message dialog appears.



4. Check whether the USB dongle icon now lights up in a vivid blue color.

Rerun the rtl_test program:

```
$ sudo rtl_test –t
```

The ouput should be as follows:

```
Found 1 device(s):
0: Realtek, RTL2838UHIDIR, SN: 00000001

Using device 0: generic RTL2843U OEM

Kernel driver is active, or device is claimed by second instance of
librtlsdr.
In the first case, please either detach or blacklist the kernel module
(dvb_usb_rtl28xxu), or enable automatic detaching at compile time.
Usb_claim_interface error -6
Failed to open rtlsdr device #0.
```

The problem is the Linux kernel DVB driver is loaded, which means it is making the device available for TV reception. Since the device is in use by this driver, the SDR programs can't access it.

Source: (Rettig, 2013)

There are two solutions. One solution is quick and temporary, just type:

```
sudo rmmod dvb_usb_rtl28xxu rtl2832
```

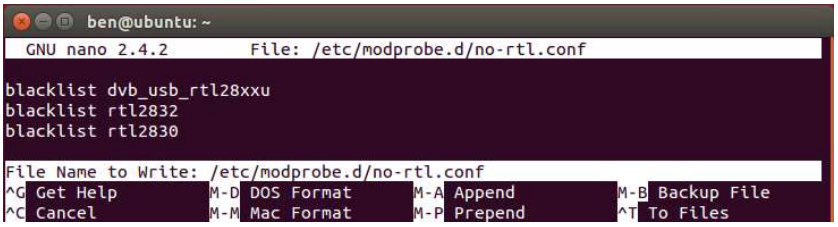
The second solution is permanently detaching the kernel from the USB dongle:

Create a file in /etc/modprobe.d/ with a .conf extension (call it "no-rtl.conf") with these contents:

```
blacklist dvb_usb_rtl28xxu
blacklist rtl2832
blacklist rtl2830
```

Nano can be used to create and save the file with the necessary content:

```
$ nano /etc/modprobe.d/no-rtl.conf
```



^O = Ctrl+O or F3 Write the current file to disk

File name to Write: /etc/modprobe.d/no-rtl.conf

[Enter]

^X = Ctrl+X (or F2) Close the current file buffer / Exit from nano

Now run:

```
$ sudo rmmod dvb_usb_rtl28xxu rtl2832
```

Now either reboot or run the following command if you get this error:

```
rmmod: ERROR: Module dvb_usb_rtl28xxu is in use
rmmod: ERROR: Module rtl2832 is in use
```

After reboot or reconnecting the USB dongle run:

```
$ sudo rmmod dvb_usb_rtl28xxu rtl2832
$ sudo rtl_test –t
```

The correctly working device should now output:

```
Found 1 device(s):
0: Realtek, RTL2838UHIDIR, SN: 00000001

Using device 0: Generic RTL2832U OEM
```

```
Found Rafael Micro R820T tuner
Supported gain values (29): 0.0 0.9 1.4 2.7 3.7 7.7 8.7 12.5 14.4 15.7 16.6
19.7 20.7 22.9 25.4 28.0 29.7 32.8 33.8 36.4 37.2 38.6 40.2 42.1 43.4 43.9
44.5 48.0 49.6
[R82XX] PLL not locked!
Sampling at 2048000 S/s.
No E4000 tuner found, aborting.
```

Appendix 3: Installation of gnuradio

Manual installation of gnuradio is not necessary when following the guidelines in Appendix 4: Installation of grgsm.

However if you insist on installing gnuradio by hand, you can run:

```
apt-get install gnuradio gnuradio-dev
```

Appendix 4: Installation of grgsm

Grgsm can be installed by two means. First there is the pybombs installation. Second there is the manual installation. It is advised to use the automated installation using pybombs.

Automated installation instructions can be found here:

https://github.com/ptrkrysik/gr-gsm/wiki/Installation

It is recommended to follow the link above for up-to-date instructions. In order to be complete in this report, the instructions are shown below too:

```
$ sudo apt-get install git python-pip
$ sudo pybombs prefix init /usr/local -a default_prx
$ sudo pybombs config default_prefix default_prx
$ sudo pybombs recipes add gr-recipes
git+https://github.com/gnuradio/gr-recipes.git
$ sudo pybombs recipes add gr-etcetera
git+https://github.com/gnuradio/gr-etcetera.git
$ sudo pybombs install gr-gsm
$ sudo ldconfig
```

Full installation takes about one hours on two cores of an i5-5675C @ 3.10GHz. Space needed for the installation is around 6.03 GiB.

Manual installation instructions can be found here:

https://github.com/ptrkrysik/gr-gsm/wiki/Manual-compilation-and-installation

It important to note that Python has a buffering problem when running grgsm_scanner. This problem interferes with the operation of this project. To

circumvent the problem edit the top line of the grgsm_scanner script. Change the first line from:

```
#!/usr/bin/python2
```

to

```
#!/usr/bin/python2 –u
```

The result is now outputted directly to STDOUT (e.g. the screen or another script) unbuffered.

Appendix 5: Installation of Wireshark

To analyse the packets generated by grgsm a program like Wireshark is necessary. For this report both Wireshark and the terminal-based version Tshark are installed. Wireshark allows for organic search on the fly, while Tshark allows for easy automation.

To install both Wireshark and Tshark run the following commands:

```
$ sudo apt-get install wireshark
$ sudo apt-get install tshark
```

Appendix 6: Visualizing frequencies

Finding frequencies is easily done by using grgsm_scanner. When further visual inspection of the spectrum is required a program like gqrx can be used.

Installation instruction can be found here: http://gqrx.dk/download/install-ubuntu

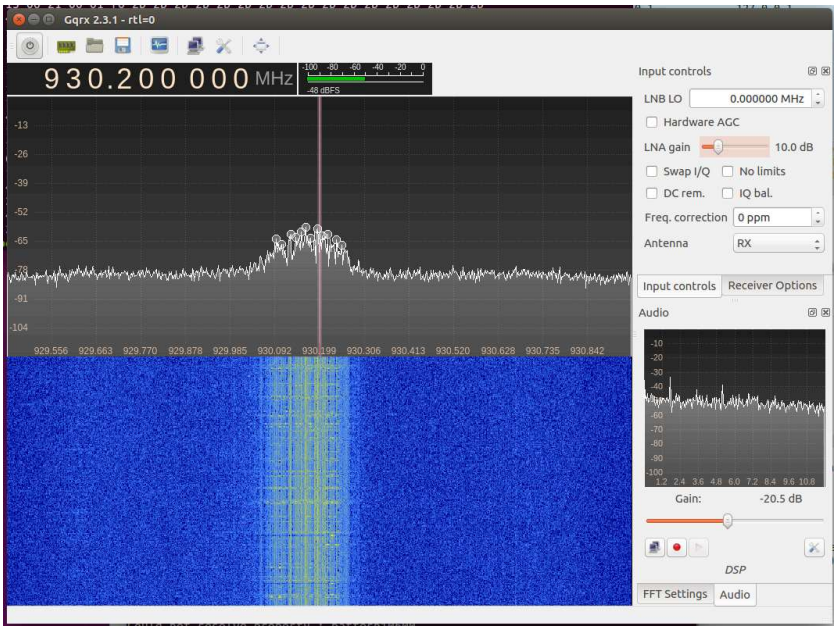
To install run the command:

```
$ sudo apgt-get install gqrx-sdr
```

To start the program run

```
$ gqrx
```

Frequencies around 930.2 MHz can easily be visualized as shown below:



Appendix 7: imsicc_scanfreq.pl

It important to note that Python has a buffering problem when running grgsm_scanner. This problem interferes with the operation of this project. To circumvent the problem edit the top line of the grgsm_scanner script. Change the first line from:

```
#!/usr/bin/python2
```

to

```
#!/usr/bin/python2 –u
```

The result is now outputted directly to STDOUT (e.g. the screen or another script) unbuffered.

The Perl script imsicc_scanfreq.pl will now work correctly.

Create a file named 'imsicc_scanfreq.pl' and paste the following code in this file:

```
#!/usr/bin/perl

############################################################
# imsicc_scanfreq.pl
# IMSI Catcher^2 or IMSI Catcher Catcher
# Script by Ben Hup
# Copyright (c) 2016 Ben Hup
# Created date 4 March 2016 v0.001
# Revised date 5 March 2016 v0.01
# Revised date 7 March 2016 v0.02
# Revised date 19 March 2016 v0.03 - by Piotr Tekieli
```

```
# Revised date 24 March 2016 v0.04 - by Andy Chiu
# Description: scan for frequencies in the GSM900 / GSM-R band for any GSM tower.
#              so real towers and IMSI-catchers; Any tower traffic will be picked up.
# Dependency: /usr/local/bin/grgsm_scanner
#
# Problem statement: Not all cell towers will be detected unless you execute 'problem solving'
# Problem solving:
#      step 1: run 'sudo nano /usr/local/bin/grgsm_scanner'
#      step 2: Change first line from: "#!/usr/bin/python2" to "#!/usr/bin/python2 -u"
#      Explanation: now Python will write unbuffered output to STDOUT, not missing any output/to
##############################################################

use strict;
use warnings;
use DBI;

my $dbfile   = "imsicc.db";
my $dsn      = "dbi:SQLite:dbname=$dbfile";
my $user     = "";
my $password = "";
my $dbh = DBI->connect($dsn, $user, $password, {
PrintError       => 0,
RaiseError       => 1,
AutoCommit       => 1,
FetchHashKeyName => 'NAME_lc',
});

my $sql = <<'END_SQL';
CREATE TABLE IF NOT EXISTS towers (
id                  INTEGER PRIMARY KEY AUTOINCREMENT,
frequency           UNSIGNED INTEGER(10),   /* 800 MHz to 1900 MHz, so 10 digits max */
arfcn               UNSIGNED INTEGER(5),
lai_mcc                 UNSINGED INTEGER(3),    /* max 3 digits: mobile country code */
lai_mnc                 UNSINGED INTEGER(3),    /* 2 to 3 digits: mobile network code */
lai_lac                 UNSINGED INTEGER(5),    /* 16 bit, thus max 5 digits: location area code
cellid              UNSINGED INTEGER(5),    /* 16 bit, thus max 5 digits: unique in lac */
neighbourcells          VARCHAR(255), /* space delimited list of neighbour ARFCN's */
signalstrength          INTEGER(4),     /* in dBm (because weak signal signed; often NEGATIVE!)
asksreauth              INTEGER(1),
selectivehandover       INTEGER(1),
nohandover              INTEGER(1),
usedencryption          TEXT,       /* ENUM("A5/0", "A5/1", "A5/2", "A5/3"), */
/* or make usedencryption a VARCHAR and paste tshark string */
nmea                    TEXT,       /* VARCHAR(255) */
nrrejects           INTEGER(7),
nrupdates           INTEGER(7),
nrciphercommands    INTEGER(7),
latitude     TEXT, /* VARCHAR(255) */
longitude TEXT, /* VARCHAR(255) */
recordadded             TEXT,       /* DATETIME */
recordrevised           TEXT,       /* DATETIME */
pcapngtower             INTEGER(1),
reselection_offset              INTEGER(7),
temporary_offset                INTEGER(7),
reselect_hysteresis INTEGER(7)
)
END_SQL
$dbh->do($sql) || die $_;

## show current content database
#my $sql = 'SELECT * FROM roguetowers';
#my $sth = $dbh->prepare($sql);
##$sth->execute();
```

```
##while (my @row = $sth->fetchrow_array) {
##    print "fname: $row[0]  lname: $row[1]\n";
##}
#$sth->execute();
#while (my $row = $sth->fetchrow_hashref) {
##    print "fname: $row->{fname}  lname: $row->{lname}\n";
#     foreach my $key (keys $row){
#           print "$key: ".$row->{$key}." ";
#     }
#     print "; ";
#}
#exit;

my $countround = 0;
print "Start scanning for cell towers (e.g. real ones and IMSI catchers)...\n";

while(1==1){ # infinite loop
# Example grgsm_scanner output
# ARFCN: 1022, Freq:  934.6M, CID: 63283, LAC:   225, MCC: 204, MNC:   4, Pwr: -27
#  |---- Configuration: 1 CCCH, not combined
#  |---- Cell ARFCNs: 59, 987, 988, 1004
#  |---- Neighbour Cells: 975, 976, 978, 979, 983, 984, 995, 999, 1000, 1004, 1005, 1006, 101

$countround++;
print '-==[ Scan round: '.$countround." ]==-\n";
print ".\n";

#Do one scan on R-GSM band and another with default option (P-GSM) for getting all towers
for (my $i=1; $i <= 2; $i++) {

my $arfcn = '';
my $signalstrength = '';
my $lai_lac = '';
my $lai_mcc = '';
my $lai_mnc = '';
my $cellid = '';
my $neighbourcells = '';
my $frequency = '';

my @scanner;
#change to default option (P-GSM) after R-GSM has been scanned
if($i == 2) {
print "-==[ Scanning with default band (P-GSM) ]==-\n";
@scanner = `/usr/local/bin/grgsm_scanner -v`;
}
else {
print "-==[ Scanning with -b R-GSM ]==-\n";
@scanner = `/usr/local/bin/grgsm_scanner -b R-GSM -v`;
}


foreach my $scanline (@scanner){
$scanline =~ s/\r|\n//gi;
my %scansplit = ();
print $scanline."\n";
if($scanline =~ m/^ARFCN\:\s/){
print '1';
%scansplit = split(/[\:\,]/, $scanline);
foreach my $key (keys %scansplit){
$scansplit{$key} =~ s/^\s+|\s+$//g; # strip spaces begin and end
$scansplit{$key} =~ s/^\t+|\t+$//g;     # strip tabs begin and end
my $trimmedkey = lc($key);
$trimmedkey =~ s/^\s+|\s+$//g;
```

```
$trimmedkey =~ s/^\t+|\t+$//g;
$scansplit{$trimmedkey} = $scansplit{$key};
}
$arfcn = $scansplit{'arfcn'};
$signalstrength = $scansplit{'pwr'};
$lai_lac = $scansplit{'lac'};
$lai_mcc = $scansplit{'mcc'};
$lai_mnc = $scansplit{'mnc'};
$cellid = $scansplit{'cid'};
$frequency = $scansplit{'freq'};

$frequency =~ s/M//gi;
$frequency = $frequency * 1000000;
}
elsif($scanline =~ m/\|---- Neighbour Cells\: /){
print '2';
my @split = split(/Neighbour Cells\:/, $scanline);
$neighbourcells = $split[1];
$neighbourcells =~ s/^\s+|\s+$//g;
#            }
#            elsif(length($scanline) <= 0){
print '3';
$dbh->do('INSERT INTO towers (frequency, arfcn, lai_mcc, lai_mnc, lai_lac, cellid, neighbourcells
undef,
$frequency, $arfcn, $lai_mcc, $lai_mnc, $lai_lac, $cellid, $neighbourcells, $signalstrength);

%scansplit = ();
$arfcn = '';
$signalstrength = '';
$lai_lac = '';
$lai_mcc = '';
$lai_mnc = '';
$cellid = '';
$neighbourcells = '';
$frequency = '';

$dbh->disconnect;
$dbh = DBI->connect($dsn, $user, $password, {
PrintError       => 0,
RaiseError       => 1,
AutoCommit       => 1,
FetchHashKeyName => 'NAME_lc',
});
}
}
}


}

$dbh->disconnect;

exit(0);
```

After creating the file imsicc_scanfreq.pl run the following commands:

```
$ chmod +x imsicc_scanfreq.pl
$ sudo apt-get install libdbi-perl
$ sudo apt-get install libcpan-sqlite-perl
```

imsicc_scanfreq.pl can now be run via:

```
$ ./ imsicc_scanfreq.pl
```

The program will run in a continuous loop scanning for frequencies and filling the SQLite database.

To exit the program hit Ctrl-C.

Appendix 8: Setting up GPS Capturing

The process of setting up GPS capturing is mostly straightforward and should not take too much time to be completed. The necessary steps and options that should be selected in order to configure a reliable connection between a GPS receiver and the scanning device (i.e. computer) are presented below.

First , a proxy application needs to be installed, so that the computer responsible for capturing GSM information could access and complement it with an appropriate format of GPS data. For this purpose a free version of ShareGPS by Jillybunch was used (see figure A). The software is available for Android devices and can be obtained through Play Store.

Having done that, a communication between the aforementioned devices needs to be established. The easiest way to do so, is by turning the mobile device into portable Wi-Fi hotspot. This way we create a separated and protected network, where each device can be reached by an IP address without the necessity to perform additional configuration procedures (see figure B). To enable capturing of GPS data, the receiver has to be enabled and set to high accuracy mode (see figure C).

Next , ShareGPS application needs to be powered up and configured for further usage. The program has an intuitive and detailed interface, which by default presents a set of information regarding actual position of the

Fig. A – ShareGPS on Android Market

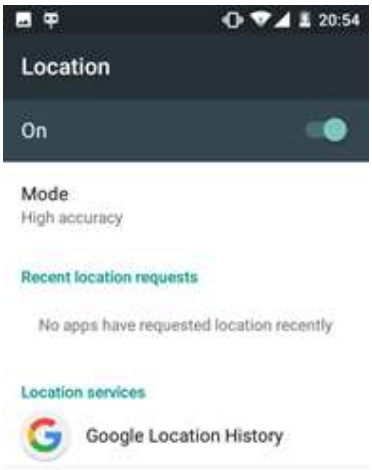


Figure B – Setting up Wi-Fi hotspot

receiving device, such as, geographical longitude, latitude, altitude and speed of traveling. For this project, only the first two parameters were necessary and, in effect, filtered out. The configuration can be done by moving to "connections" tab and creating a new profile inside the section. In order to do so, the "add" button needs to be clicked (see figure D). The aforementioned IP infrastructure approach can be obtained by setting appropriate options shown on the figure below (see figures E,F). With the created profile, all what is left to do on the mobile device limits to clicking on newly established position. The status of such should change from "idle" or "disconnected" to "listening" (see figure G).

Figure C – Enabling GPS receiver

Figure D – Connections tab

Figure E – IP Profile - Step 1

Figure F – IP Profile – Step 2

At this moment, the computer can be connected to mobile device through Wi-Fi link. It is essential to check both devices' IP addresses and compere them to have a clear image on whether each of them belongs to the same subnet. This may differ if a user uses virtual machines equipped with Virtual Network Interfaces set to be working in NAT translation mode as a part of testing environment. In that case, the change of adapter operation setting (from NAT to bridging) may be necessary and vary from one virtualization platform to another. In case, the situation is clear and both devices can be reached (e.g. with ping command), the GPS data capturing application can be run (see next chapter). The active exchange of data is indicated with a status "connected" in ShareGPS (see figure H).

Figure G – Listering status

Figure H – Connected status

Appendix 9: GetGPS (JDK 7)

The following console application introduces a new functionality for capturing and storing the most recent position of scanning device. This kind of data has a significant importance for a proper conductance of triangulation processes on a set of found based stations. The software is capable of capturing a stream of NMEA sequences (Mehaffey, 2016) generated by GPS receiver and shared through HTTP protocol. The newest version supports a manual provision of current location, in case of signal loss during measurements done in environments with high level of its attenuation (such as concrete buildings).

According to the assumptions of Object Oriented Programming, this application was divided into three classes, each fulfilling a separate role in the whole process. **GPSA** class is a core, invoking functions from two remaining classes. Its responsibility consists of getting initial parameters, such us GPS device's IP, communication port and database location, running check functions (whether the aforementioned parameters are correct or not), and delivering final data to the database, through the invocation of a set of SQL queries. **ConnectionTester** class is responsible for testing whether the communication with GPS device can be

performed or not. **DBConnection**'s duty is to enable communication with the selected SQLite database.

The software is compatible and has been tested with Java Development Kit v7. It can also be compiled with newer JDK versions, however, due to compatibility reasons with OpenJDK environment of Ubuntu 15.10, the recent recommendation is not to do so yet.

```
package com.tudelft.hacking.getgps;

import java.io.*;
import java.net.*;
import java.text.DecimalFormat;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;
import java.util.Scanner;

/**
*
* @author Piotr Tekieli <p.s.tekieli@student.tudelft.nl>
* Last revised : 16.04.2016
*/
public class GPSA {
private static int gpsshare_port = 0;
private static String gpsshare_ipaddress = "0.0.0.0";
private static int timeout = 15000;
private static String nmea = "";
private static double latitude = 0;
private static String latitudefull = "";
private static double longitude = 0;
private static String longitudefull = "";
private static boolean locked = true;

public static void main(String[] args) {
boolean controller = false;
do {
System.out.println("Select appropriate option: ");
System.out.println("1 - Automatic capture, 2 - Manual provision");
Scanner reader = new Scanner(System.in);
String text = reader.nextLine();

switch (text) {
case "1" :
Automatic(args[0], args[1]);
controller = true;
break;
case "2" :
Manual();
controller = true;
break;
default :
System.out.println("Wrong option! Select again!");
break;
}
} while (controller == false);

System.out.println("Phase 3/3 - Saving data to the database... ");
DBConnection db = new DBConnection(args[2]);
String adate = GetDate();
```

```
int[] counters = new int[4];
counters[0] = db.UpdateStatement("UPDATE towers SET recordadded='" + adate + "' WHERE nn
counters[1] = db.UpdateStatement("UPDATE towers SET latitude='" + latitudefull + "' WHERE nn
counters[2] = db.UpdateStatement("UPDATE towers SET longitude='" + longitudefull + "' WHERE
counters[3] = db.UpdateStatement("UPDATE towers SET nmea='" + nmea + "' WHERE nmea IS
db.CloseConnection();
System.out.println(" ...Done!");

int sum = 0;
for (int i : counters)
sum += i;
System.out.println(sum + " fields updated!");
}

private static void Automatic(String IP, String PORT) {
ConnectionTester ctr = new ConnectionTester();

System.out.print("Phase 1/3 - Checking the communication Computer <-> GPS Device... ");
if ( ctr.validIP(IP) != true ) {
System.out.println("The provided IP address is not valid");
System.exit(100);
}
gpsshare_ipaddress = IP;

if ( ctr.validPort(PORT) != true ) {
System.out.println("The provided PORT number is not valid");
System.exit(101);
}
gpsshare_port = Integer.parseInt(PORT);

if ( ctr.TestRemotePort(gpsshare_ipaddress, gpsshare_port, timeout) != true) {
System.out.println("The software was not able to verify the connection. Try again later!");
System.exit(102);
}
System.out.println(" ...Done!");

System.out.println("Phase 2/3 - Checking the stream properties and getting NMEA data... ");
CheckAndGetNMEA();
System.out.println(" ...Done!");
}

private static void Manual() {
Scanner reader = new Scanner(System.in);
System.out.println("Enter latitude [xx.xxxxxN]");
latitudefull = reader.nextLine();
latitudefull = latitudefull.replace(".", ",");
System.out.println("Enter longitude [x.xxxxxE]");
longitudefull = reader.nextLine();
longitudefull = longitudefull.replace(".", ",");
nmea = "NO_INFO";
}

private static void CheckAndGetNMEA() {
try {
System.out.println("Waiting for synchronization... [5s]");
Socket reader = new Socket();
try {
Thread.sleep(5000);
} catch (InterruptedException ex) {
System.out.println("Error while trying to execute sleep instruction!" + ex);
}
reader.connect(new InetSocketAddress(gpsshare_ipaddress, gpsshare_port), timeout);
BufferedReader in = new BufferedReader(new InputStreamReader(reader.getInputStream()));
```

```
String inputLine;
long timer = System.currentTimeMillis();
long endtimer = timer + 25000;
System.out.println("Waiting for GPGGA String... [45s]");
while ( System.currentTimeMillis() < endtimer && locked) {
inputLine = in.readLine();
//System.out.println(inputLine); //For Debugging Purposes
if(inputLine.startsWith("$GPGGA")) {
String delims = "[,]";
String[] tokens = inputLine.split(delims);
if (tokens[2].replace(".","") != null && tokens[4].replace(".","") != null) {
System.out.println("$GPGGA string found : ");
System.out.println(inputLine);
double latitude_p1 = ( Double.parseDouble(tokens[2].substring(0,2)) );
double latitude_p2 = ( (Double.parseDouble( ( tokens[2].replace(".","") ).substring(2) ) / Math.p
double longitude_p1 = ( Double.parseDouble(tokens[4].substring(2,3)) );
double longitude_p2 = ( (Double.parseDouble( ( tokens[4].replace(".","") ).substring(3) ) / Math
DecimalFormat rounded = new DecimalFormat("##.######");
System.out.println("Are the calculated values correct?");
System.out.println("Latitude: " + rounded.format( latitude_p1 + latitude_p2 ) + tokens[3] + " Lo
System.out.print("Press ENTER to continue or CTRL+C to exit ");
KeyPressed();
nmea = inputLine;
latitude = latitude_p1 + latitude_p2;
longitude = longitude_p1 + longitude_p2;
latitudefull = rounded.format( latitude ) + tokens[3];
longitudefull = rounded.format( longitude ) + tokens[5];
locked = false;
}
}
}
if (locked != false) {
System.out.println("Error getting NMEA data. Try again!");
System.exit(103);
}
in.close();
reader.close();
} catch (IOException io_ex) {
System.out.println("Problem with executing BufferReader section... Try again!" + io_ex);
}
}

private static String GetDate() {
Date now = new Date( );
SimpleDateFormat df = new SimpleDateFormat("MMM d, yyy HH:mm:ss.SSSSSSSSS", Locale.EN
return df.format(now);
}

private static void KeyPressed() {
Scanner scan = new Scanner(System.in);
scan.nextLine();
}
}
```

```
package com.tudelft.hacking.getgps;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
```

```
/**
*
* @author Piotr Tekieli <p.s.tekieli@student.tudelft.nl>
*/
public class DBConnection {

private Connection toDB = null;
private String DBPath = "imsicc.db"; //Default path

public DBConnection(String source) {
SetDBPath(source);
CreateConnection();
}

private void CreateConnection() {
try {
Class.forName("org.sqlite.JDBC");
toDB = DriverManager.getConnection("jdbc:sqlite:" + DBPath);
} catch ( ClassNotFoundException | SQLException db_ex ) {
System.err.println( db_ex.getClass().getName() + ": " + db_ex.getMessage() );
System.exit(110);
}
}

public void CloseConnection() {
try {
toDB.close();
} catch (SQLException sql_ex) {
System.out.println("Error closing connection with the database. " + sql_ex);
System.exit(112);
}
}

private void SetDBPath(String path) {
DBPath = path;
}

public int UpdateStatement(String sql) {
try {
Statement st = toDB.createStatement();
int rowcount = st.executeUpdate(sql);
st.close();
return rowcount;
} catch (SQLException sql_ex) {
System.out.println("Error while performing I/O operations on the database. " + sql_ex);
System.exit(111);
}
return 0;
}
}
```

```
package com.tudelft.hacking.getgps;

import java.net.InetSocketAddress;
import java.net.Socket;

/**
*
* @author Piotr Tekieli <p.s.tekieli@student.tudelft.nl>
```

```
*/
public class ConnectionTester {
protected boolean TestRemotePort(String gpsshare_ipaddress, int gpsshare_port, int timeout) {
try {
Socket testsocket = new Socket();
testsocket.connect(new InetSocketAddress(gpsshare_ipaddress, gpsshare_port), timeout);
testsocket.close();
return true;
} catch (Exception port_ex) {
System.out.println("Error while testing remote port: " + port_ex);
return false;
}
}

/* Reused from (source) : http://stackoverflow.com/a/5240291 */
protected boolean validIP (String ip) {
try {
if ( ip == null || ip.isEmpty() ) {
return false;
}
String[] parts = ip.split( "\\." );
if ( parts.length != 4 ) {
return false;
}
for ( String s : parts ) {
int i = Integer.parseInt( s );
if ( (i < 0) || (i > 255) ) {
return false;
}
}
if ( ip.endsWith(".") ) {
return false;
}

return true;
} catch (NumberFormatException nfe) {
System.out.println("Stage 1 error [IPV4_FORMAT]: " + nfe);
return false;
}
}

protected boolean validPort(String portToparse) {
try {
int ParsedPort = Integer.parseInt(portToparse);
if ( (ParsedPort < 0) || (ParsedPort > 65535) )
return false;
return true;
}
catch (NumberFormatException nfe) {
System.out.println("Stage 1 error [PORT_FORMAT]: " + nfe);
return false;
}
}
}
```

In order to execute GetGPS application, enter the following command

```
$ java –jar GPSA.jar GPS_IP COMMUNICATION_PORT DATABASE_LOCATION
```

and replace three last variables with an appropriate data. After that, you can either select to run the software in an automatic mode, which enforces communication with the GPS device for the recovery of NMEA data, or run the procedure manually and enter all the necessary data by yourself.

Appendix 10: DBMerger (JDK 7)

DBMerger is an application aimed to resolve problems with merging local SQLite databases. As the result of a specific selection of technologies used in this project, a problem with connecting gathered information into one single source has occurred. Each member, equipped with his own capturing devices, produced at least one SQLite database during the whole duration of the project (usually more than that). It naturally resulted in numerous problems during manual merging operations, which needed to be solved with an automated solution. This is an answer to that issue. Note: This code presents the "backend" of DBMerger. Since it was designed as GUI application, the author has decided to omit the description of graphical part which is of no relevance to the project.

```
package com.tud.hacklab.dbmerger;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.ArrayList;

/**
*
* @author Piotr Tekieli <p.s.tekieli@student.tudelft.nl>
* Revised : 19.04.2016
*/
public final class DBHandle {

private Connection baseDB = null;
private Connection assignDB = null;
private final ArrayList<String> Files = new ArrayList<>();

public DBHandle(String base, String append) {
Files.add(base);
Files.add(append);
CreateConnection();
MergeDatabases();
CloseConnection();
}
private void CreateConnection() {
try {
Class.forName("org.sqlite.JDBC");
baseDB = DriverManager.getConnection("jdbc:sqlite:" + Files.get(0));
assignDB = DriverManager.getConnection("jdbc:sqlite:" + Files.get(1));
} catch ( ClassNotFoundException | SQLException db_ex ) {
System.err.println( db_ex.getClass().getName() + ": " + db_ex.getMessage() );
System.exit(110);
}
}
private void CloseConnection() {
try {
baseDB.close();
```

```
assignDB.close();
} catch (SQLException sql_ex) {
System.out.println("Error closing connection with the database. " + sql_ex);
System.exit(112);
}
}
private void MergeDatabases() {
try {
assignDB.prepareStatement("CREATE TABLE towers2 (\n" +
"       id INTEGER,\n" +
"       frequency UNSIGNED INTEGER(10),    /* 800 MHz to 1900 MHz */\n" +
"       arfcn UNSIGNED INTEGER(5),\n" +
"       lai_mcc UNSINGED INTEGER(3), /* max 3 digits: mobile country code */\n" +
"       lai_mnc     UNSINGED INTEGER(3),     /* 2 to 3 digits: mobile network code */\n" +
"       lai_lac       UNSINGED INTEGER(5),     /* location area code */\n" +
"       cellid UNSINGED INTEGER(5),     /* unique in lac */\n" +
"       neighbourcells VARCHAR(255),    /* space delimited list of neighbour ARFCN */\n" +
"       signalstrength    INTEGER(4),      /* in dBm */\n" +
"       asksreauth INTEGER(1),\n" +
"       selectivehandover INTEGER(1),\n" +
"       nohandover INTEGER(1),\n" +
"       usedencryption TEXT,         /* ENUM(\"A5/0\", \"A5/1\", \"A5/2\", \"A5/3\"), */\n" +
"       nmea TEXT,       /* VARCHAR(255) */\n" +
"       signalgain INTEGER(4),\n" +
"       nrrejects INTEGER(7),\n" +
"       nrupdates INTEGER(7),\n" +
"       nrciphercommands INTEGER(7),\n" +
"       latitude TEXT, /* VARCHAR(255) */\n" +
"       longitude TEXT, /* VARCHAR(255) */\n" +
"       recordadded TEXT,      /* DATETIME */\n" +
"       recordrevised TEXT,    /* DATETIME */\n" +
"       pcapngtower INTEGER(1)\n" +
");").execute();
assignDB.prepareStatement("INSERT INTO towers2 SELECT * FROM towers;").execute();
assignDB.prepareStatement("UPDATE towers2 SET id=NULL;").execute();
baseDB.prepareStatement("ATTACH DATABASE \"" + Files.get(1) + "\" AS  fromDB;").execute();
baseDB.prepareStatement("INSERT INTO towers SELECT * FROM fromDB.towers2;").execute();
assignDB.prepareStatement("DROP TABLE IF EXISTS towers2;").execute();
} catch (SQLException sql_ex) {
System.out.println("Error inserting data. " + sql_ex);
System.exit(113);
}
}
}
```

Problem !

The selected approach (of creating an auxiliary table during transfer operation) was caused by the occurrence of multiple identical primary keys in both databases. Since SQLite does not support an elastic removal of PK constraints from tables that are already created, without performing such “duplication” process, we were not able to simply nullify the ID column (in an auxiliary database) and copy the entries as new ones to the “global” DB. Instead of that, we cloned the primary table of the auxiliary database while omitting the aforementioned constrain and duplicated data to that place. Such data was no longer resistant to the nullifications, so we filled all rows of ID column with NULL values. The newly prepared table was later merged with a final one. The order of entries remained the same.

The picture below demonstrates how DBMerger's GUI looks like:

Appendix 11: config.py

Configuration file used by other scripts. It contains variables necessary for capturing, post processing and giveaway detection.

```
other_saving_location = '/media/andy/CaptureStorage/latest'  # set a (valid!) location here. The
frequencies = []  # as can be found through grgsm_livemon  # as can be found through grgsm_l
frequencies_scanner = [945.2] #932.0, 933.8, 936.4, 938.0, 938.4]  # as can be found through
test_frequencies = True
capture_length = '120'  # in seconds
number_of_rounds = 1  # number of captures to be done (this number * capture length = total s
delete_capture_after_processing = False

decode_bcch = True  # one of these 2 (or both) must be True, otherwise nothing happens:)
decode_sdcch = True
execute_decode = True  # if set to False then no decoding is done (at all, ignoring settings above

use_wireshark = False  # False = use tshark

available_antennas = ['00000006']
######### POST PROCECCSING VARIABLES ###########
one_file_mode = None  # Process one file or multiple
db_location = '/media/andy/CaptureStorage/latest' + '/imsicc.db'


######### GIVEAWAY VARIABLES ##########
#A5/5 = 4, #A5/4 = 3 #A5/3 = 2, #A5/2 = 1, A5/1 = 0 A5/0 = A5/0
allowedEncryption = [-1,0,2]
updates_minimum = 30 # minimum number of update requests seen before the rejection giveaw
rejection_ratio = 1.25  #number of update requests / number of cipher commands
```

Appendix 12: main.py

Python script for performing captures after active frequencies have been found and set in the config file.

```
import os
```

```
import tempfile
import threading
from Queue import Queue
from os.path import expanduser
import time
from subprocess import Popen, PIPE
import thread
import sys
import shlex

import config
from colorama import init, Fore, Back, Style
init(autoreset=True)


class Capturing:
def __init__(self):
"""
Start capturing GSM packets and decode them
Print statements used:
Green = Normal operation, information
Yellow = Executing statements
Red = Something went wrong
"""
if not os.geteuid() == 0:
sys.exit("\nYou must be root to run this application, please use sudo and try again.\n")
self.continue_loop = True

print(Fore.GREEN + '------------- Imsi Catcher^2 -------------')
print(Fore.GREEN + 'To stop the loop: Hit Enter')

# set used location
if config.other_saving_location:
user_home_location = config.other_saving_location
else:
user_home_location = expanduser("~")
location = user_home_location + "/IMSI/captures"
print(Fore.GREEN + 'Saving location: ' + location)

# make save folder if it does not exist yet. Change bash location to new folder.
if not os.path.exists(location):
os.makedirs(location)
os.chdir(location)

# make a queue for decodes to be done
self.stop_decode = False
self.q = Queue()

# start wireshark or tshark:
if config.use_wireshark:
wiresharkBashCommand = "sudo wireshark -k -f udp -Y gsmtap -i lo"
print(Fore.YELLOW + 'Executing command: ' + str(wiresharkBashCommand))
Popen(wiresharkBashCommand, shell=True)
time.sleep(5)  # sleep to allow you to hit enter for the warning messages :)

# make separate folder for this capture (with current time)
foldername = time.strftime("%d-%m-%Y_%H:%M:%S")
os.makedirs(foldername)
os.chdir(foldername)

#  device queue
self.deviceq = Queue()
for device in config.available_antennas:
```

```
self.deviceq.put(device)

# allow entry of frequencies in both formats
self.frequencies = []
temp_frequencies = []
for freq in config.frequencies:
temp_frequencies.append(freq)
for short_freq in config.frequencies_scanner:
long_freq = str(int(short_freq * 1000000))
temp_frequencies.append(long_freq)

# test frequencies if they are really alive
if config.test_frequencies:
results = {}
for freq in temp_frequencies:
working_freq = self.test_frequency(freq)
results[freq] = working_freq
for freq, working in results.iteritems():
print 'freq: ' + str(freq) + ', working: ' + str(working)
if working:
self.frequencies.append(freq)
else:
self.frequencies = temp_frequencies

# print 'The following frequencies will be processed:'
# for freq in self.frequencies:
#     print freq

# start actually doing stuff
self.start_loop()

def start_loop(self):
"""
Run the capturing and decoding until one presses a key and the enter key. The loop will finish it's
"""
stop = []
thread.start_new_thread(self.stop_loop, (stop,))
if config.execute_decode:
thread.start_new_thread(self.decode_loop, ())
capture_thread = None
i = 0
while not stop:
# capture certain amount of times
if i < config.number_of_rounds:
# every freq in one iteration
for freq in self.frequencies:
while True:
# only continue if antenna available
if self.deviceq.qsize() > 0:
# TODO: add check which sees if enough harddiskspace
antenna = str(self.deviceq.get())
print(Fore.GREEN + 'starting capturing on freq ' + str(freq))

capture_thread = threading.Thread(target=self.capture_raw_data, args=(i, freq, antenna))
capture_thread.start()
time.sleep(2)
break
else:
time.sleep(10)
i += 1
else:
break
```

```
# keep thread alive for last capture, and then wait for decode loop to die
flag = 1
while flag:
flag = capture_thread.isAlive()
time.sleep(3)

self.stop_decode = True
print(Fore.GREEN + 'Finished ALL capturing')
if not self.q.empty() and config.execute_decode:
self.decode_loop

def decode_loop(self):
"""
Decode GSM Data to GSM packets with use of a queue (which contains the filenames to decode)
"""
while True:
# check if we should stop (so finished capturing, and the decode queue is empty
if self.stop_decode and self.q.empty():
print(Back.YELLOW + 'stop_decode = true')
break
# if the queue is filled, we should decode that!
if not self.q.empty():
filename, freq = self.q.get()
self.decode_raw_data(filename, freq)
# if the queue is empty, let's chill for a while
else:
time.sleep(5)

print(Fore.GREEN + 'Finished ALL decoding; let\'s go to to the next location!')

def capture_raw_data(self, filenumber, freq, antenna):
"""
Capture raw antenna data using grgsm_capture.
"""
filename = 'capture' + str(filenumber) + '_' + str(freq) + '.cfile'
captureBashCommand = "grgsm_capture.py -c " + filename + " -f " + freq + ' -T ' \
+ config.capture_length \
+ ' --args="rtl=' + antenna + '"'
#captureBashCommand = 'grgsm_capture.py -c %s -f %d -T %s  --args="rtl=%s"' % (
#     filename, freq, config.capture_length, antenna
# )
print(Fore.YELLOW + 'Executing command: ' + str(captureBashCommand))
print(Fore.GREEN + 'Script will capture for ' + config.capture_length + ' seconds.')

# Start capture
os.system(captureBashCommand)

everything_to_hell = False
# Check if capture was succesful (as sometimes receiver quits for no reason)
if not os.path.isfile(filename):
print(Back.RED + str(filename) + ': I think the capture went wrong, please check!! I will not deco
everything_to_hell = True

print(Fore.GREEN + str(filename) + ': Finished capturing')
if not everything_to_hell:
print(Back.YELLOW + 'adding to queue: ' + str(filename))
self.q.put((filename, freq))
self.deviceq.put(antenna)

def decode_raw_data(self, filename, freq):
"""
Decode the captured data into GSM packets readable by wireshark. Also deleted raw data when r
"""
```

```
print(Fore.GREEN + str(filename) + ': Starting decoding of SDCCH8 and BCCH.')
SDCCH_bash = 'grgsm_decode -c ' + filename + ' -f ' + freq + ' -m SDCCH8 -t 1'
BCCH_bash = 'grgsm_decode -c ' + filename + ' -f ' + freq + ' -m BCCH -t 0'

# Start Tshark if wanted to capture this packet output of this capture file
if not config.use_wireshark:
tsharkBashCommand = "sudo tshark -w " + filename[:-6] + ".pcapng -i lo -q"
args = shlex.split(tsharkBashCommand)
print(Fore.YELLOW + 'Executing command: ' + str(tsharkBashCommand))
tshark = Popen(args)
#print(Back.RED + 'tshark should have started')
time.sleep(5)  # sleep to allow you to enter sudo password

# Sleep to allow release of lock on file
time.sleep(2)

# Actually start decoding, depending on which channel to decode
if config.decode_sdcch:
print(Fore.YELLOW + str(filename) + ': Decoding SDCCH, using commmand: ' + SDCCH_bash)
SDCCH = Popen(SDCCH_bash, shell=True)
SDCCH.wait()
if config.decode_bcch:
print(Fore.YELLOW + str(filename) + ': Decoding BCCH, using commmand: ' + BCCH_bash)
BCCH = Popen(BCCH_bash, shell=True)
BCCH.wait()

# Delete if wanted
if config.delete_capture_after_processing:
print(Fore.GREEN + str(filename) + ': Deleting capture file as requested (change this in config fil
os.remove(filename)

if not config.use_wireshark:
tshark.terminate()
#print(Back.RED + 'tshark should have stopped')

print(Fore.GREEN + str(filename) + ': Finished decoding')

def stop_loop(self, stop):
"""
Stops the loop in start_loop. The loop finishes and is then ended.
"""
raw_input()
stop.append(None)
print(Fore.RED + 'The processing will stop after finishing current capturing and decoding...')

def test_frequency(self, frequency):
"""
Test the given frequency with grgsm_livemon, by checking if there's a lot of output in the termina
"""
p = Popen(['grgsm_livemon', '-f', frequency], stdout=PIPE)
i = 0
found = False
print(Fore.GREEN + 'Testing frequency: ' + str(frequency))

# loop through lines of terminal output
while True:
line = p.stdout.readline()

# skip first 15 lines for false positives
if i > 15:
if '2b 2b' in line:
found = True
print(Back.GREEN + 'Working: ' + str(frequency))
```

```
break
# Now something should have happened, if not..quit
if i > 120:
break
i += 1
p.wait()
# sleep for reattaching kernel driver again.
time.sleep(3)
return found

Capturing()
while 1:
time.sleep(5)
```

Appendix 13: post_processing.py

Extract useful information from .pcapng capture files and store it in the local database

```
import os
from os.path import expanduser
import sys
import pyshark
import sqlite3
import config
from colorama import init, Fore, Back, Style
init(autoreset=True)

class PostProcessing:
def __init__(self):
"""
Post process the resulting pcapng files; look for giveaways (and input this in the DB with BST info
Look for every pcapng file in IMSI folder and then process per frequency. at the end put this info
"""
# prepare files to process. If only one file; enter this in the config. otherwise search imsi folder re
filelist = []
if config.one_file_mode:
filelist.append(config.one_file_mode)
else:
# set used location
if config.other_saving_location:
user_home_location = config.other_saving_location
else:
user_home_location = expanduser("~")
location = user_home_location + "/IMSI/captures"

# search for correct files
for dirpath, dirnames, filenames in os.walk(location):
for filename in (f for f in filenames if f.endswith(".pcapng")):
filelist.append(os.path.join(dirpath, filename))

# connect to the -already existing- database, see https://docs.python.org/2/library/sqlite3.html
if not os.path.isfile(config.db_location):
print(Fore.RED + 'Error: DB file does not exist, check given db_location in config. Exiting now')
sys.exit()
self.connection = sqlite3.connect(config.db_location)
self.cursor = self.connection.cursor()
```

```
# rejection dictionary
self.rejections = {}
self.reject_per_freq_counter = 0
self.location_updating_request_per_freq_counter = 0
self.accept_per_freq_counter = 0

self.other_data = {}

# post-process all pcapng files
for location in filelist:
frequency = self.get_freq_by_filename(location)
print '\n Now processing frequency: ' + str(frequency) + ' using file: ' + os.path.basename(locati
cell_id, cipher, reselect_offset, temporary_offset, reselect_hysteris = self.process_capture(locatio
print 'cell id: ' + str(cell_id)

if cell_id == -1:
print(Fore.RED + 'WARNING: no cell id detected for frequency ' + str(frequency))
print(Style.RESET_ALL)
if cipher == -1:
print(Fore.RED + 'WARNING: no cipher detected for frequency ' + str(frequency))
print(Style.RESET_ALL)

if cipher == 55: #translate no cipher int to str
cipher = "A5/0"

# save rejections table
rejec, request, accept = self.rejections.get((cell_id, frequency), [0, 0, 0])
new_num_rejec = rejec + self.reject_per_freq_counter
new_num_request = request + self.location_updating_request_per_freq_counter
new_num_accept = accept + self.accept_per_freq_counter
self.rejections[(cell_id, frequency)] = [new_num_rejec, new_num_request, new_num_accept]

self.other_data[(cell_id, frequency)] = [reselect_offset, temporary_offset, cipher, reselect_hyster

# reset counters
self.reject_per_freq_counter = 0
self.location_updating_request_per_freq_counter = 0
self.accept_per_freq_counter = 0
print 'cipher: ' + str(cipher)
print 'reselect offset: ' + str(reselect_offset)
print 'temporary offset: ' + str(temporary_offset)
print 'reselect hysteris: ' + str(reselect_hysteris)

print '\n Rejections/requests/accepts numbers: '
print self.rejections

print 'Processing this data into the database...'

# start adding to database
for cell_id, frequency in self.rejections:
rejects, requests, accepts = self.rejections[(cell_id, frequency)]
reselect_offset, temporary_offset, cipher, reselect_hysteris = self.other_data[(cell_id, frequency)

# First, check if the combination of cellid and freq (which makes it unique) already exists in the D
content = (cell_id, frequency)
self.cursor.execute('SELECT * FROM towers WHERE cellid=? AND frequency=?', content)
result = self.cursor.fetchone()

# Then insert depending on result
content = (rejects, requests, accepts, cell_id, frequency, cipher, reselect_offset, temporary_offse
#if result is None:  # combination of cellid and freq doesn't exist yet, make a new row
```

```
#Whether a row already exists for combination of cellid and freq, always insert new row so not o
self.cursor.execute('INSERT INTO towers (nrrejects, nrupdates, nrciphercommands, cellid, freque

# And finally actually apply changes made
self.connection.commit()

# close db connection when finished
self.connection.close()

print '\n all done:)'

def process_capture(self, location):
"""
Process the given pcapng file; for every packet in this capture, check what it is and call accompa
"""
capture = pyshark.FileCapture(location)
print capture
i = 0

# initialise values for if it isn't found in the packets
cell_id = 0
new_cipher = -1
reselect_offset = 0
temporary_offset = 0
reselect_hysteris = 0
for packet in capture:
i += 1

# all ccch traffic
if hasattr(packet, 'gsm_a.ccch'):
# Sys info 3
#print packet['gsm_a.ccch'].gsm_a_dtap_msg_rr_type
if packet['gsm_a.ccch'].gsm_a_dtap_msg_rr_type == hex(27):
#print 'hello sys3'
cell_id, reselect_offset, temporary_offset, reselect_hysteris = self.process_sys_info_3(packet)

# lapdm cipher traffic
if hasattr(packet, 'gsm_a.dtap'):
cipher = self.process_lapdm_ciphering_mode(packet)

# check if there are multiple ciphers in 1 freq (first one is used)
if new_cipher == -1 and cipher != -1:
#print new_cipher
new_cipher = cipher
if new_cipher != -1 and cipher != new_cipher and cipher != -1:
if int(str(cipher)) not in config.allowedEncryption:
print(Fore.RED + 'Detected unsafe Encryption:' + str(cipher))
new_cipher = cipher
print 'number of packets processed: ' + str(i)
return int(str(cell_id), 16), int(str(new_cipher), 16), int(str(reselect_offset), 16), int(str(tempora

def process_sys_info_3(self, packet):
"""
Process sys info packets, giving:
'e212_mcc', 'e212_mnc', 'field_names', 'get_field', 'get_field_by_showname', 'get_field_value',
'gsm_a_bssmap_cell_ci', 'gsm_a_dtap_msg_rr_type', 'gsm_a_l3_protocol_discriminator', 'gsm_a
'gsm_a_rr_acc', 'gsm_a_rr_acs', 'gsm_a_rr_att', 'gsm_a_rr_bs_ag_blks_res', 'gsm_a_rr_bs_pa_
'gsm_a_rr_cbq', 'gsm_a_rr_cbq3', 'gsm_a_rr_ccch_conf', 'gsm_a_rr_cell_barr_access',
'gsm_a_rr_cell_reselect_hyst', 'gsm_a_rr_cell_reselect_offset', 'gsm_a_rr_dtx_bcch', 'gsm_a_rr_
'gsm_a_rr_l2_pseudo_len', 'gsm_a_rr_max_retrans', 'gsm_a_rr_ms_txpwr_max_cch', 'gsm_a_rr
'gsm_a_rr_penalty_time', 'gsm_a_rr_pwrc', 'gsm_a_rr_radio_link_timeout', 'gsm_a_rr_re',
'gsm_a_rr_rxlev_access_min', 'gsm_a_rr_si13_position', 'gsm_a_rr_t3212', 'gsm_a_rr_temporar
'gsm_a_rr_tx_integer', 'gsm_a_skip_ind', 'layer_name', 'pretty_print', 'raw_mode']
```

```
"""
ccch = packet['gsm_a.ccch']
lac = ccch.gsm_a_lac
cell_id = ccch.gsm_a_bssmap_cell_ci
mcc = ccch.e212_mcc
mnc = ccch.e212_mnc
reselect_offset = ccch.gsm_a_rr_cell_reselect_offset
temporary_offset = ccch.gsm_a_rr_temporary_offset # if needed
reselect_hysteris = ccch.gsm_a_rr_cell_reselect_hyst
return cell_id, reselect_offset, temporary_offset, reselect_hysteris

def process_lapdm_ciphering_mode(self, packet):
"""
Process the lapdm packets, which give the ciphering command, possible field names:

'e212_mcc', 'e212_mnc', 'field_names',
'follow_on_request', 'get_field', 'get_field_by_showname', 'get_field_value', 'gsm_a_a5_1_algorit
'gsm_a_es_ind', 'gsm_a_ie_mobileid_type', 'gsm_a_l3_protocol_discriminator', 'gsm_a_lac', 'gsm
'gsm_a_msc_rev', 'gsm_a_oddevenind', 'gsm_a_rf_power_capability', 'gsm_a_skip_ind', 'gsm_a_
'gsm_a_spareb8', 'gsm_a_tmsi', 'gsm_a_unused', 'layer_name', 'msg_mm_type', 'pretty_print',
'protocol_discriminator', 'raw_mode', 'seq_no', 'updating_type']
"""
cipher = -1
dtap = packet['gsm_a.dtap']
# check if this packet has DTAP Radio Resources Management Message Type: Ciphering Mode Co
if hasattr(dtap, 'msg_rr_type'):
# if message type is that of ciphering mode command
if dtap.msg_rr_type == '0x35': #53

self.accept_per_freq_counter += 1
try:
cipher = dtap.gsm_a_rr_algorithm_identifier
except AttributeError:
print(Fore.RED + '\nNO ENCRYPTION PRESENT!\n')
print(Style.RESET_ALL)
cipher = 37 #means no encryption, decimal: 55

# federico rejection stuff
if hasattr(dtap, 'msg_mm_type'):
# DTAP Mobility Management Message Type: Location Updating Reject (0x04) == dtap.msg_mm_
if dtap.msg_mm_type == '0x04':
self.reject_per_freq_counter += 1

# DTAP Mobility Management Message Type: Location Updating Request (0x08)
if dtap.msg_mm_type == '0x08':
self.location_updating_request_per_freq_counter += 1
return cipher

def get_freq_by_filename(self, filename):
"""
Frequency has to be determined for database access. split by _, take last section, remove file ext
"""
return filename.split('_')[-1][:-7]


PostProcessing()
```

Appendix 14: giveaways.py

Look for multiple giveaways in a single database file. There is a function for each giveaway that returns a list of (cellid, frequency) pairs

```
import os
import sqlite3
import sys
import config
from colorama import init, Fore, Back, Style
init(autoreset=True)


class Giveaways:
def __init__(self):
"""
Look for multiple giveaways in a single database file which is
there is a function for each giveaway that returns a list of (cellid, frequency) pairs
"""

#Check if database exists
if not os.path.isfile(config.db_location):
print(Fore.RED + 'Error: DB file does not exist, check given db_location in config. Exiting now')
sys.exit()
self.towersList = getUniqueTowers()

self.timestamps = getTimestamps()


############## GIVEAWAYS ###############
#returns a list of towers that use one of the encryption methods specified in the config file
def encryption(self):
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
towers = set()
query = 'SELECT DISTINCT cellid, frequency FROM towers WHERE cellid != 0'
#check config file for selected encryptions to filter on
#modify query if any encryption methods are specified
if config.allowedEncryption:
query += ' AND '
encryptionTypes = set()
for encryption in config.allowedEncryption:
encryptionTypes.add('usedencryption != ' + str(encryption))
query += " AND ".join(encryptionTypes)

for row in cursor.execute(query):
towers.add((row[0],row[1]))

connection.close()
return towers

def neighbourList(self):
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
flaggedTowers = set()

approvedTowers = set() #Maintain a set of towers that have found neighbours accross all timesta
#Checking for the giveaway is done for towers found with the same timestamp
for t in self.timestamps:
cellinfo = set() #for storing cell information
for row in cursor.execute('SELECT DISTINCT arfcn, neighbourcells, cellid, frequency FROM towers
cellinfo.add((row[0], row[1], row[2], row[3]))

#For each neighbour list, check if at least one tower in the list is found with the same timestamp
```

```
for neighbourlist in cellinfo:
if neighbourlist[2] != 0: #cell id should be present, otherwise information not reliable
foundNeighbour = False
for arfcn in cellinfo:
if str(arfcn[0]) in neighbourlist[1]:
foundNeighbour = True
approved = (neighbourlist[2], neighbourlist[3])
if approved not in approvedTowers:
approvedTowers.add(approved)
if not foundNeighbour:
flaggedTowers.add((neighbourlist[2], neighbourlist[3]))

towers = flaggedTowers - approvedTowers #remove towers that have been approved at least onc
connection.close()
return towers

#TODO gain giveaway
def gain(self):
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
towers = set()

query = 'SELECT DISTINCT cellid, frequency FROM towers WHERE reselection_offset != 0 OR tem

for row in cursor.execute(query):
towers.add((row[0],row[1]))

connection.close()
return towers

#TODO rejection giveaway
def rejections(self):
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
towers = set()

query = 'SELECT DISTINCT cellid, frequency, nrrejects, nrupdates, nrciphercommands FROM tow
for row in cursor.execute(query):
nrrejects = row[2]
nrupdates = row[3]
nrciphercommands = row[4]
#The number of rejections should not be significant compared to the sum of rejections and ciphe
if (nrrejects + nrciphercommands) > config.updates_minimum:
if nrrejects > ((nrrejects + nrciphercommands) * config.rejection_ratio):
towers.add((row[0],row[1]))

connection.close()
return towers


#Retrieve a list of all unique towers. Can check each tower for a specific giveaway in a later stag
def getUniqueTowers():
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
towers = []
for row in cursor.execute('SELECT DISTINCT cellid, frequency FROM towers WHERE cellid != 0'):
towers.append((row[0],row[1]))
connection.close()
return towers

#Retrieve a list of all unique timestamps for detecting the neighbour list giveaway
def getTimestamps():
connection = sqlite3.connect(config.db_location)
```

```
cursor = connection.cursor()
timestamps = []
#First get all distinct timestamps, checking for the neighbouring list giveaway is done per scan
for row in cursor.execute('SELECT DISTINCT recordadded FROM towers WHERE cellid != 0 AND a
timestamps.append(row[0])
connection.close()
return timestamps


giveaway = Giveaways()
```

Appendix 15: IMSIcatcher.py

This script creates a new table where only (cellid, frequency) pairs are listed that have at least one giveaway. The table provides a clear overview of which giveaways are present. Each giveaway function from giveaways.py is processed here separately to create the new table. The table serves as a clear overview of possible IMSIcatchers.

```
import os
import sqlite3
import sys
import config
import giveaways as ga
import csv
from colorama import init, Fore, Back, Style
init(autoreset=True)


class IMSIcatcher:
def __init__(self):
"""
This script creates a new table where only (cellid, frequency) pairs are listed that have at least or
The table provides a clear overview of which giveaways are present
Each giveaway function from giveaways.py is processed here separately to create the new table
Furthermore, the locations of possible imsicatchers are written to a .csv file used for triangulatior
"""

connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()

cursor.execute('DROP TABLE IF EXISTS imsicatchers')
cursor.execute("""CREATE TABLE imsicatchers (
id                  INTEGER PRIMARY KEY AUTOINCREMENT,
cellid              UNSINGED INTEGER(5),
frequency  UNSIGNED INTEGER(10),
encryption VARCHAR (255), /* encryption used (wireshark value) */
neighbourlist UNSIGNED INTGEGER (5), /* stores TRUE if giveaway is present */
signal_gain         VARCHAR (255), /* tuple (reselection offset, temporary offset) */
rejections  VARCHAR (255)) /* triple (#rejections, #updates, #ciphercommands) */ """)

processEncryption()
processNeighbourList()
processGain()
processRejections()
towerCount = 0
```

```
cursor.execute('SELECT * FROM IMSIcatchers')
response = cursor.fetchall()
csvdata = [['cellid', 'frequency', 'signalstrength', 'longitutde', 'latitude']] #data to be written to cs
for tower in response:
towerCount += 1
print 'Cell ID: ' + redIfTrue(tower[1])
print 'Frequency: ' + redIfTrue(tower[2])
print 'Encryption Giveaway: ' + redIfTrue(tower[3])
print 'Neighbour list Giveaway: ' + redIfTrue(tower[4])
print 'Signal Gain Giveaway: ' + redIfTrue(tower[5])
print 'Rejections Giveaway: ' + redIfTrue(tower[6])
locCount = 1;
for location in cursor.execute('SELECT DISTINCT latitude, longitude, signalstrength FROM towers
print 'Location ' + str(locCount) + ': ' + str(location[0]) + ', ' + str(location[1])
locCount += 1
csvdata.append([tower[1], tower[2], location[2], location[0], location[1]])
if locCount == 1:
print 'Location not available'

print ' '

print Fore.RED + str(towerCount) + ' possible IMSIcatchers found!'
print 'Data saved to database'

connection.commit()
connection.close()

#write csv data to file
with open('locations.csv', 'w') as fp:
a = csv.writer(fp, delimiter=',')
a.writerows(csvdata)



def processEncryption():
encryptionTowers = giveaways.encryption()
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()

for tower in encryptionTowers:
for result in encryptionTowers:
cellid = tower[0]
frequency = tower[1]
cursor.execute("SELECT cellid FROM imsicatchers WHERE cellid = ? AND frequency = ?", (cellid, f
data = cursor.fetchone()

cursor.execute('SELECT usedencryption FROM towers WHERE cellid = ? AND frequency = ? AND u
cipher = cursor.fetchone()[0]
if data is None:
cursor.execute("INSERT INTO imsicatchers (cellid, frequency, encryption) VALUES (?,?,?)", (cellid
else:
cursor.execute("UPDATE imsicatchers SET encryption = ? WHERE cellid = ? AND frequency = ?",
connection.commit()
connection.close()

def processNeighbourList():
neighbourListTowers = giveaways.neighbourList()
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
for tower in neighbourListTowers:
cellid = tower[0]
frequency = tower[1]
cursor.execute("SELECT cellid FROM imsicatchers WHERE cellid = ? AND frequency = ?", (cellid, f
```

```
data = cursor.fetchone()
cursor.execute('SELECT DISTINCT latitude, longitude FROM towers WHERE cellid = ? AND freque
locCount = len(cursor.fetchall())
if data is None:
cursor.execute("INSERT INTO imsicatchers (cellid, frequency, neighbourlist) VALUES (?,?,?)", (cel
else:
cursor.execute("UPDATE imsicatchers SET neighbourlist = 'TRUE' WHERE cellid = ? AND frequenc
connection.commit()
connection.close()

def processGain():
gainTowers = giveaways.gain()
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
for tower in gainTowers:
cellid = tower[0]
frequency = tower[1]
cursor.execute("SELECT cellid FROM imsicatchers WHERE cellid = ? AND frequency = ?", (cellid, f
data = cursor.fetchone()
cursor.execute("SELECT reselection_offset, temporary_offset, reselect_hysteresis FROM towers W
gain_data = cursor.fetchone()
signal_gain = str(gain_data[0]) + ' ' + str(gain_data[1]) + ' ' + str(gain_data[2])

if data is None:
cursor.execute("INSERT INTO imsicatchers (cellid, frequency, signal_gain) VALUES (?,?,?)", (cellic
else:
cursor.execute("UPDATE imsicatchers SET signal_gain = ? WHERE cellid = ? AND frequency = ?",
connection.commit()
connection.close()

def processRejections():
rejectionTowers = giveaways.rejections()
connection = sqlite3.connect(config.db_location)
cursor = connection.cursor()
for tower in rejectionTowers:
cellid = tower[0]
frequency = tower[1]
cursor.execute("SELECT cellid FROM imsicatchers WHERE cellid = ? AND frequency = ?", (cellid, f
data = cursor.fetchone()
cursor.execute("SELECT nrrejects, nrupdates, nrciphercommands FROM towers WHERE cellid = ?
rejection_data = cursor.fetchone()
rejection = str(rejection_data[0]) + ' ' + str(rejection_data[1]) + ' ' + str(rejection_data[2])

if data is None:
cursor.execute("INSERT INTO imsicatchers (cellid, frequency, rejections) VALUES (?,?,?)", (cellid,
else:
cursor.execute("UPDATE imsicatchers SET rejections = ? WHERE cellid = ? AND frequency = ?", (
connection.commit()
connection.close()

def redIfTrue(value):
if (value != None ):
return Fore.RED + str(value)
else:
return str(value)



giveaways = ga.Giveaways()
IMSIcatcher()
```

Appendix 16: create_ciphercommands.py

Script for creating a text file that exhausts all possible combinations of ciphering mode commands. A pcapng can be created afterwards using the text2pcap program.

```
#hexadecimal representation of a ciphering command package
#only the 45th byte will be modified containing the cipher mode.
cipher_template1 = """0000   00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00
0010   00 43 41 fd 40 00 40 11 fa aa 7f 00 00 01 7f 00
0020   00 01 e9 a8 12 79 00 2f fe 42 02 04 01 01 00 00
0030   dc 00 00 0f 54 6d 08 00 01 00 03 64 0d 06 35 """

cipher_template2 = """
0040   59 71 d1 8e d9 39 45 b9 c5 b1 55 ca 26 b6 ce 4e
0050   6e
"""

cipherString = ""

#ciphering mode is set using 4 bits
for i in range(0,16):
ciphermode = '{0:02x}'.format(i)
cipherString = cipherString + cipher_template1 + ciphermode + cipher_template2

#write generated strings to file
with open("ciphertext", "w") as text_file:
text_file.write(cipherString)
```

Article changelog

- 2018-04-04: conversion of DOCX and manual HTML 5, image and CSS conversion.
- 2016-05-31: Final document delivered to Dr. Christian Doerr at the faculty Electrical Engineering, Mathematics & Computer Science, department Cyber Security. The final hard copy consisted of 15747 words. The original title is "The battle against GSM eavesdropping: Detecting IMSI-catchers". The original document was part of the course IN4253ET Hacking-Lab.

Latest educational
KLM Flight Academy selectieprocedure

Latest travel
Zuid-Afrika, adventure of a lifetime

Latest FreeBSD
Kopia backup using Nextcloud WebDAV

+ Sitemap - Explore BenHup.com

BENHUP.COM

Sitemap
Terms & Disclaimer
Privacy statement
Contact Ben Hup

PROFESIONAL

Mission
Education
Courses
Certificates
Skills
Experience

PERSONAL

Personal life
Video montage

SUBSCRIBE

Newsletter
RSS Feed

SOCIAL MEDIA

LinkedIn of Ben Hup
Twitter